

Discrete Mathematics

Part III. Mathematical Logic

Xuming He

Associate Professor

School of Information Science and Technology
ShanghaiTech University

Spring Semester, 2024

非划定考试范围

Course Evaluation

- Your feedback is valuable for us to improve the course. Thanks for the time and effort.



Proposition

Proposition: a declarative sentence that is either true or false.

- simple, compound, propositional constant/variable

Logical Connectives: $\neg$ (unary), $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$ (binary)

- Truth table

Well-Formed Formulas (WFFs): formulas

- propositional constants (T, F) and propositional variables are WFFs
- $\neg A$, $(A \wedge B)$, $(A \vee B)$, $(A \leftarrow B)$, $(A \leftrightarrow B)$
- Finite

Precedence (priority):

- ① Formulas inside () are computed firstly
- ② Different connectives: $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$ (Decreasing Precedence)
- ③ Same connectives: from left to the right

Types of WFFs

Tautology_(重言式): a WFF whose truth value is **T** for all truth assignment

- $p \vee \neg p$ is a tautology

Contradiction_(矛盾式): a WFF whose truth value is **F** for all truth assignment

- $p \wedge \neg p$ is a contradiction

Contingency_(可能式): neither tautology nor contradiction

- $p \rightarrow \neg p$ is a contingency

Satisfiable_(可满足的): a WFF is satisfiable if it is true for at least one truth assignment

Rule of Substitution_(代入规则): Let B be a formula obtained from a tautology

A by substituting a propositional variable in A with an arbitrary formula. Then B must be a tautology.

- $p \vee \neg p$ is a tautology: $(q \wedge r) \vee \neg(q \wedge r)$ is a tautology as well.

From Natural Language to WFFs

The Method of Translation:

- Introduce symbols $p, q, r \dots$ to represent simple propositions
- Connect the symbols with logical connectives to obtain WFFs

Remark: it is better to choose the simple proposition to be affirmative sentence.

DEFINITION: Let F be a WFF of $p_1, \dots, p_n$, n propositional variables

- A **truth assignment** (真值指派) for F is a map $\alpha: \{p_1, \dots, p_n\} \rightarrow \{\mathbf{T}, \mathbf{F}\}$.
 - There are 2^n different truth assignments.

Logically Equivalent

DEFINITION: Let A and B be WFFs in propositional variables $p_1, \dots, p_n$.

- A and B are **logically equivalent** (等值) if they always have the same truth value for every truth assignment (of $p_1, \dots, p_n$)
 - Notation: $A \equiv B$

THEOREM: $A \equiv B$ if and only if $A \leftrightarrow B$ is a tautology.

- $A \equiv B$
- iff for any truth assignment, A, B take the same truth values
- iff for any truth assignment, $A \leftrightarrow B$ is true
- iff $A \leftrightarrow B$ is a tautology

THEOREM: $A \equiv A$; If $A \equiv B$, then $B \equiv A$; If $A \equiv B, B \equiv C$, then $A \equiv C$

QUESTION: How to prove $A \equiv B$?

Proving $A \equiv B$

Method 1: Show that A, B have the same truth table.

Method 2: Rule of Replacement: (替换规则) Replacing a sub-formula in a formula F with a logically equivalent sub-formula gives a formula logically equivalent to the formula F .

Method 3. THEOREM: Let $A^{-1}(\mathbf{T})$ be the set of truth assignments such that A is true.

- $A \equiv B$ if and only if $A^{-1}(\mathbf{T}) = B^{-1}(\mathbf{T})$.
- $A \equiv B$ if and only if $A^{-1}(\mathbf{F}) = B^{-1}(\mathbf{F})$

Tautological Implications

DEFINITION: Let A and B be WFFs in propositional variables $p_1, \dots, p_n$.

- A **tautologically implies** (重言蕴涵) B if every truth assignment that causes A to be true causes B to be true.
 - Notation: $A \Rightarrow B$, called a **tautological implication**
 - $A^{-1}(\mathbf{T}) \subseteq B^{-1}(\mathbf{T}); B^{-1}(\mathbf{F}) \subseteq A^{-1}(\mathbf{F})$

THEOREM: $A \Rightarrow B$ iff $A \rightarrow B$ is a tautology.

- $A \Rightarrow B$ iff $A^{-1}(\mathbf{T}) \subseteq B^{-1}(\mathbf{T})$ iff $A \rightarrow B$ is a tautology

THEOREM: $A \Rightarrow B$ iff $A \wedge \neg B$ is a contradiction.

- $A \rightarrow B \equiv \neg A \vee B \equiv \neg(A \wedge \neg B)$

Proving $A \Rightarrow B$: (1) $A^{-1}(\mathbf{T}) \subseteq B^{-1}(\mathbf{T});$ (2) $B^{-1}(\mathbf{F}) \subseteq A^{-1}(\mathbf{F});$

(3) $A \rightarrow B$ is a tautology; (4) $A \wedge \neg B$ is a contradiction

Argument

DEFINITION: An **argument** (论证) is a sequence of propositions

- **Conclusion** (结论): the final proposition
- **Premises** (假设): all the other propositions
- **Valid** (有效): the truth of premises implies that of the conclusion
- **Proof** (证明): a valid argument that establishes the truth of a conclusion

DEFINITION: An **argument form** (论证形式) is a sequence of formulas.

- **Valid** (有效): no matter which propositions are substituted for the propositional variables, the truth of conclusion follows from the truth of premises
- **Rules of inference** (推理规则): valid argument forms (relatively simple)

Building Arguments

QUESTION: Given the premises $P_1, \dots, P_n$, show a conclusion Q , that is, show that $P_1 \wedge \dots \wedge P_n \Rightarrow Q$.

Name	Operations
Premise	Introduce the <u>given formulas</u> $P_1, \dots, P_n$ in the process of constructing proofs.
Conclusion	Quote the <u>intermediate formula</u> that have been deducted.
Rule of replacement	Replace a formula with a <u>logically equivalent</u> formula.
Rules of Inference	Deduct a new formula with a <u>tautological implication</u> .
Rule of substitution	Deduct a formula from a <u>tautology</u> .

Predicate and Individual

Predicate_{谓词}: describe the property of the subject term (in a sentence)

- A predicate is a function from a domain of individuals to $\{\mathbf{T}, \mathbf{F}\}$
- **n -ary predicate** _{n 元谓词}: a predicate on n individuals
 - I : “is an integer” // unary
 - G : “is greater than” //binary
- **Predicate constant**_{谓词常项}: a concrete predicate // I, G
- **Predicate variable**_{谓词变项}: a symbol that represents any predicate

Individual_{个体词}: the object you are considering (in a sentence)

- “ $\sqrt{1 + 2\sqrt{1 + 3\sqrt{1 + \dots}}}$ is an integer”
- “ e^π is greater than π^e ”
 - **Individual Constant**_{个体常项}: $\sqrt{1 + 2\sqrt{1 + 3\sqrt{1 + \dots}}}$, e^π, π^e
 - **Individual Variable**_{个体变项}: x, y, z
 - **Domain**_{个体域}: the set of all individuals in consideration

From Predicates to Propositions

Propositional function 命题函数: $P(x_1, \dots, x_n)$, where P is an n -ary predicate

- $P(x, y)$: “ x is greater than y ”
- $P(x, y)$ gives a proposition when we assign values to x, y
 - $P(e^\pi, \pi^e)$ is a proposition (a true proposition)
- $P(x, y)$ is not a proposition

EXAMPLE: p : “Alice’s father is a doctor”; q : “Bob’s father is a doctor”

- Individuals: Alice’s father, Bob’s father; Predicate D : “is a doctor”
- $p = D(\text{Alice's father})$, $q = D(\text{Bob's father})$

Function of Individuals: a map on the domain of individuals

- $f(x) = x$ ’s father
- $p = D(f(\text{Alice}))$; $q = D(f(\text{Bob}))$

Universal Quantifier

DEFINITION: Let $P(x)$ be a propositional function. The **universal quantification**_{全称量化} of $P(x)$ is “ $P(x)$ for all x in the domain”.

- notation: $\forall x P(x)$; read as “for all $x P(x)$ ” or “for every $x P(x)$ ”
 - “ $\forall$ ” is called the **universal quantifier**_{全称量词}
 - “ $\forall x P(x)$ ” is true iff $P(x)$ is true for every x in the domain
 - “ $\forall x P(x)$ ” is false iff there is an x_0 in the domain such that $P(x_0)$ is false

DEFINITION: Let $P(x)$ be a propositional function. The **existential quantification**_{存在量化} of $P(x)$ is “there is an x in the domain such that $P(x)$ ”

- notation: $\exists x P(x)$; read as “for some $x P(x)$ ” or “there is an x s.t. $P(x)$ ”
 - “ $\exists$ ” is called the **existential quantifier**_{存在量词}
 - “ $\exists x P(x)$ ” is true iff there is an x in the domain such that $P(x)$ is true
 - “ $\exists x P(x)$ ” is false iff $P(x)$ is false for every x in the domain

Well-Formed Formulas

DEFINITION: well-formed formulas 合式公式 / formulas

- 1) propositional constants, propositional variables, and propositional functions without connectives are WFFs
- 2) If A is a WFF, then $\neg A$ is also a WFF
- 3) If A, B are WFFs and there is no individual variable x which is bound in one of A, B but free in the other, then $(A \wedge B), (A \vee B), (A \rightarrow B), (A \leftrightarrow B)$ are WFFs.
- 4) If A is a WFF with a free individual variable x , then $\forall x A, \exists x A$ are WFFs.
- 5) WFFs can be constructed with 1)-4).
 - Example: $\forall x F(x) \vee G(x), \forall x P(y)$ are not WFFs
 - Example: $\exists x (A(x) \rightarrow \forall y B(x, y))$ is a WFF

Precedence: $\forall, \exists$ have higher precedence than $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$

- $\forall x P(x) \rightarrow Q(y)$ means $(\forall x P(x)) \rightarrow Q(y)$, not $\forall x (P(x) \rightarrow Q(y))$

From Natural Language to WFFs

The Method of Translation:

- Introduce symbols to represent propositional constants, propositional variables, individual constants, individual variables, predicate constants, predicate variables, functions of individuals
- Construct WFFs with 1)-4) such that WFFs reflect the real meaning of the natural language

Interpretation

DEFINITION: an **interpretation**_{解释} requires one to (remove all uncertainty)

- assign a concrete proposition to every **proposition variable**
- assign a concrete predicate to every **predicate variable**
- restrict the domain of every **bound individual variable**
- assign a concrete individual to every **free individual variable**
- choose a concrete **function**, if there is any

Types of WFFs

DEFINITION: A WFF is **logically valid** 普遍有效 if it is **T** in every interpretation

- $\forall x (P(x) \vee \neg P(x))$ is logically valid

DEFINITION: A WFF is **unsatisfiable** 不可满足 if it is **F** in every interpretation

- $\exists x (P(x) \wedge \neg P(x))$ is unsatisfiable

DEFINITION: A WFF is **satisfiable** 可满足 if it is **T** in some interpretation

- $\forall x (x^2 > 0)$
 - true when domain= nonzero real numbers

THEOREM: Let A be any WFF. A is logically valid iff $\neg A$ is unsatisfiable.

Rule of Substitution: Let A be a tautology in propositional logic. If we substitute any propositional variable in A with an arbitrary WFF from predicate logic, then we get a logically valid WFF.

- $p \vee \neg p$ is a tautology; hence, $P(x) \vee \neg P(x)$ is logically valid

Logical Equivalence

DEFINITION: Two WFFs A, B are **logically equivalent**_{等值} if they always have the same truth value in every interpretation.

- notation: $A \equiv B$; example: $\forall x P(x) \wedge \forall x Q(x) \equiv \forall x (P(x) \wedge Q(x))$

THEOREM: $A \equiv B$ iff $A \leftrightarrow B$ is logically valid.

THEOREM: $A \equiv B$ iff $A \rightarrow B$ and $B \rightarrow A$ are both logically valid.

Rule of Substitution

METHOD: Applying the rule of substitution to the logical equivalences in propositional logic, we get logical equivalences in predicate logic.

Tautological Implication

DEFINITION: Let A and B be WFFs in predicate logic. A **tautologically implies** (重言蕴涵) B if every interpretation that causes A to be true causes B to be true.

- notation: $A \Rightarrow B$, called a **tautological implication** (重言蕴涵)

THEOREM: $A \Rightarrow B$ iff $A \rightarrow B$ is logically valid.

- $A \Rightarrow B$
- iff every interpretation that causes A to be true causes B to be true
- iff there is no interpretation such that $(A, B) = (\mathbf{T}, \mathbf{F})$
- Iff $A \rightarrow B$ is true in every interpretation
- iff $A \rightarrow B$ is logically valid

THEOREM: $A \Rightarrow B$ iff $A \wedge \neg B$ is unsatisfiable.

- $A \rightarrow B \equiv \neg A \vee B \equiv \neg(A \wedge \neg B)$

Building Arguments

QUESTION: Given the premises $P_1, \dots, P_n$, show a conclusion Q , that is, show that $P_1 \wedge \dots \wedge P_n \Rightarrow Q$.

Name	Operations
Premise	Introduce the <u>given formulas</u> $P_1, \dots, P_n$ in the process of constructing proofs.
Conclusion	Quote the <u>intermediate formula</u> that have been deducted.
Rule of replacement	Replace a formula with a <u>logically equivalent</u> formula.
Rules of Inference	Deduct a new formula with a <u>tautological implication</u> .
Rule of substitution	Deduct a formula from a <u>tautology</u> .

Rules of Inference for $\forall, \exists$

Name	Rules of Inference	NO.
Universal Instantiation 全称量词消去	$\forall x P(x) \Rightarrow P(a)$ a is <u>any</u> individual in the domain of x	1
Universal Generalization 全称量词引入	$P(a) \Rightarrow \forall x P(x)$ a <u>takes any</u> individual in the domain of x	2
Existential Instantiation 存在量词消去	$\exists x P(x) \Rightarrow P(a)$ a is a <u>specific</u> individual in the domain of x	3
Existential Generalization 存在量词引入	$P(a) \Rightarrow \exists x P(x)$ a is a <u>specific</u> individual in the domain of x	4

Discrete Mathematics

Part IV. Graph Theory

Xuming He

Associate Professor

School of Information Science and Technology
ShanghaiTech University

Spring Semester, 2024

非划定考试范围

Graph

DEFINITION: A **graph** $G = (V, E)$ is defined by a nonempty set V of **vertices**_{顶点} and a set E of **edges**_边, where each edge is associated with one or two vertices (called **endpoints**_{端点} of the edge).

- **Infinite Graph**_{无限图}: $|V| = \infty$ or $|E| = \infty$
- **Finite Graph**_{有限图}: $|V| < \infty$ and $|E| < \infty$; $|V|$ is called the **order**_{阶数} of G

Types of Graphs

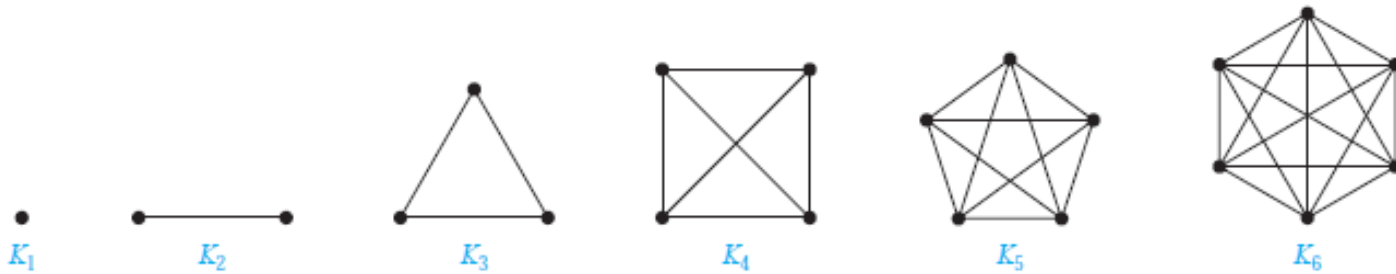
DEFINITION: Let $G = (V, E)$ be a graph with vertex set $V = \{v_1, \dots, v_n\}$.

- **Question 1:** are the edges of G **directed**有向的?
 - No: G is an **undirected graph**无向图; the edge connecting v_i, v_j : $\{v_i, v_j\}$
 - Yes: G is a **directed graph**有向图; the edge starting at v_i and ending at v_j : (v_i, v_j)
- **Question 2:** are there **multiple edges**多重边 connecting two different vertices v_i, v_j ?
 - No: G is a **simple graph**简单图; Yes: G is a **multigraph**多重图
- **Question 3:** are there **loops**自环 connecting a vertex v_i to itself?
 - Yes: G is a **pseudograph**伪图

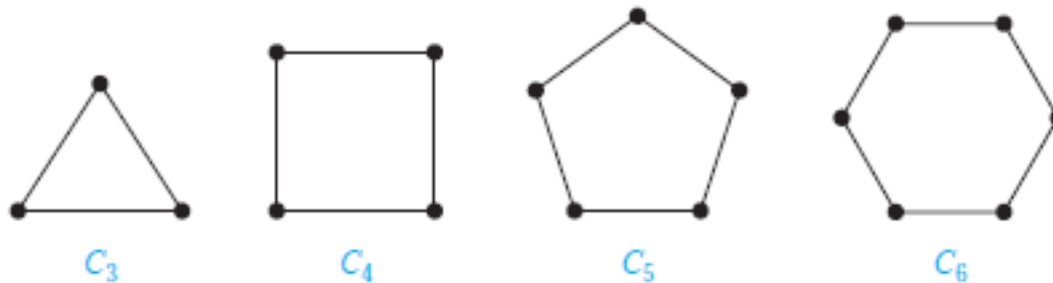
Type	Edges	Multiple Edges Allowed?	Loops Allowed?
Simple graph	undirected	No	No
Multigraph	undirected	Yes	No
Pseudograph	undirected	Yes	Yes
Simple directed graph	directed	No	No
Directed multigraph	directed	Yes	Yes
Mixed graph	undirected + directed	Yes	Yes

Special Simple Graphs

Complete Graph 完全图 K_n : $V = \{v_1, \dots, v_n\}$; $E = \{\{v_i, v_j\} : 1 \leq i \neq j \leq n\}$

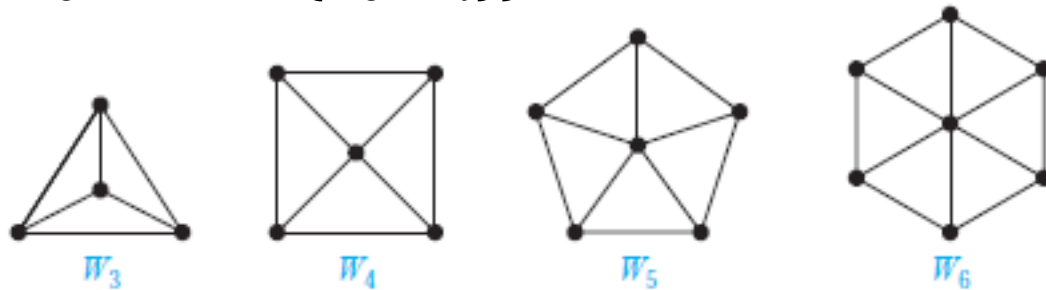


Cycle 环, 圈 C_n : $V = \{v_1, v_2, \dots, v_n\}$; $E = \{\{v_1 v_2\}, \{v_2, v_3\}, \dots, \{v_n, v_1\}\}$



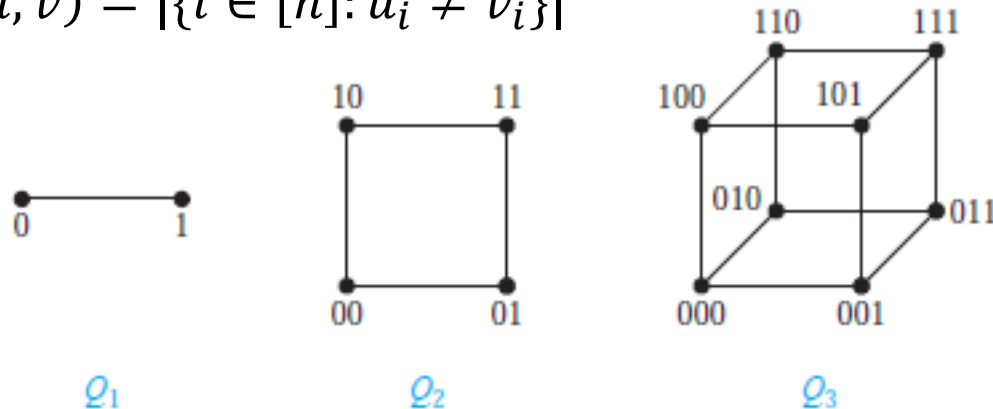
Special Simple Graphs

Wheel_轮 W_n : $V = \{v_0, v_1, v_2, \dots, v_n\}$; $E = \{\{v_1, v_2\}, \dots, \{v_n, v_1\}\} \cup \{\{v_0, v_1\}, \dots, \{v_0, v_n\}\}$



n -Cubes_{方体} Q_n : $V = \{0,1\}^n$; $E = \{\{u, v\} : d(u, v) = 1\}$

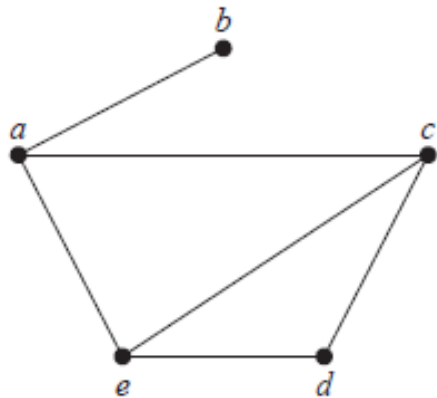
- $d(u, v) = |\{i \in [n] : u_i \neq v_i\}|$



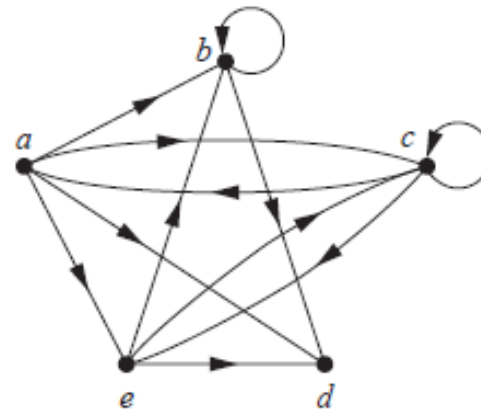
Adjacency List

DEFINITION: Let $G = (V, E)$ be a graph with no multiple edges. The **adjacency list**_{邻接表} of G is a list the vertices of the graph and all adjacent vertices

- $v_i, v_j \in V$ are **adjacent**_{相邻的} if $\{v_i, v_j\}$ or (v_i, v_j) is an edge



<i>a</i>	<i>b, c, e</i>
<i>b</i>	<i>a</i>
<i>c</i>	<i>a, d, e</i>
<i>d</i>	<i>c, e</i>
<i>e</i>	<i>a, c, d</i>

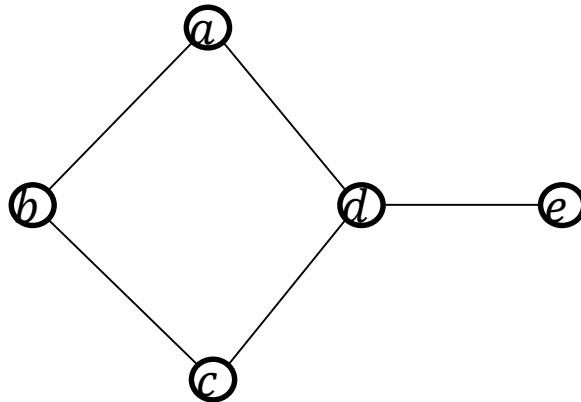


<i>a</i>	<i>b, c, d, e</i>
<i>b</i>	<i>b, d</i>
<i>c</i>	<i>a, c, e</i>
<i>d</i>	
<i>e</i>	<i>b, c, d</i>

Adjacency Matrix

DEFINITION: Let $G = (V = \{v_1, \dots, v_n\}, E)$ be a simple graph. The **adjacency matrix**_{邻接矩阵} of G is an $n \times n$ matrix $A = (a_{ij})$, where

$$a_{ij} = \begin{cases} 1 & \{v_i, v_j\} \in E \\ 0 & \{v_i, v_j\} \notin E \end{cases}$$



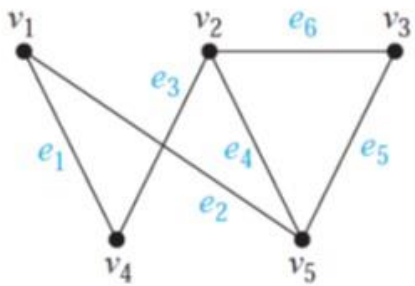
	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>a</i>	0	1	0	1	0
<i>b</i>	1	0	1	0	0
<i>c</i>	0	1	0	1	0
<i>d</i>	1	0	1	0	1
<i>e</i>	0	0	0	1	0

Incidence Matrix

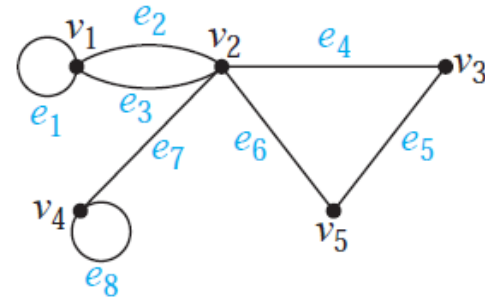
DEFINITION: Let $G = (V = \{v_1, \dots, v_n\}, E = \{e_1, \dots, e_m\})$ be undirected. The **incidence matrix** 关联矩阵 of G is an $n \times m$ matrix $B = (b_{ij})$, where

$$b_{ij} = \begin{cases} 1 & \text{if } e_j \text{ is incident with } v_i \\ 0 & \text{otherwise} \end{cases}$$

- e_j **incident with** v_i : v_i is an endpoint of e_j



$$\begin{matrix} & e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{matrix} & \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

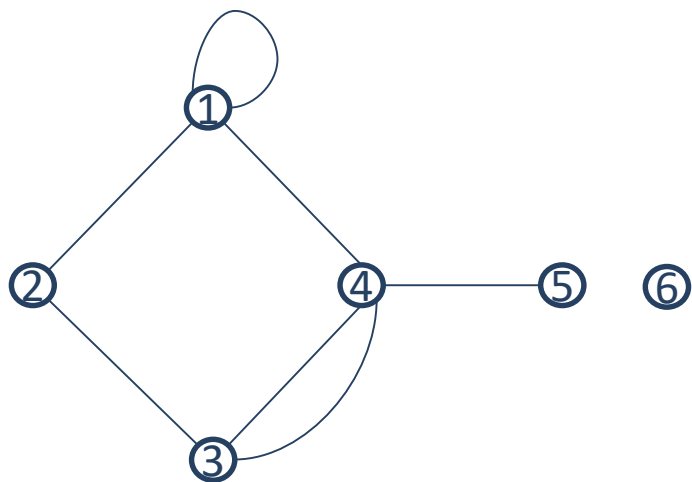


$$\begin{matrix} & e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 & e_8 \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

Degree

DEFINITION: Let $G = (V, E)$ be an undirected graph. We say that two vertices $u, v \in V$ are **adjacent**_{相邻的} (or **neighbors**_{邻居}) if $\{u, v\} \in E$.

- **neighborhood**_{邻域} of v in G : $N(v) = \{u \in V : \{u, v\} \in E\}$
 - $N(A) = \bigcup_{v \in A} N(v)$ for $A \subseteq V$
- the **degree**_度 $\deg(v)$ of $v \in V$ in G , is the number of edges incident with v
 - every loop from v to v contributes 2 to $\deg(v)$
- v is **isolated**_{孤立的} if $\deg(v) = 0$; v is **pendant**_{悬挂的} if $\deg(v) = 1$

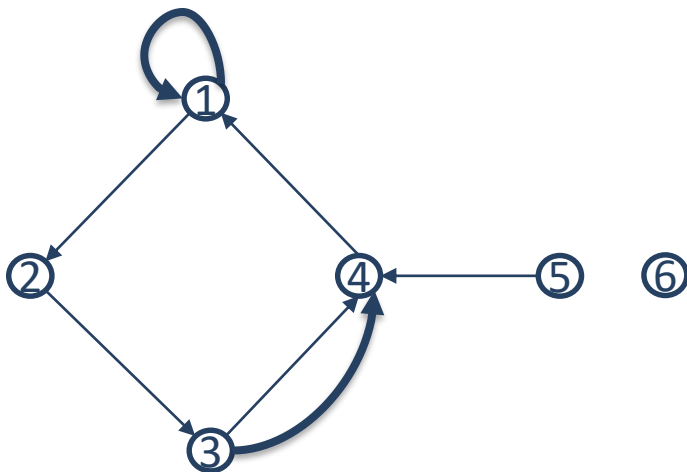


- 4 and 5 are adjacent
- $\{4, 5\}$ is incident with 4 and 5
- $N(4) = \{1, 3, 5\}$; $N(\{1, 4\}) = \{1, 2, 3, 4, 5\}$
- $\deg(1) = 4$, $\deg(2) = 2$, $\deg(3) = 3$, $\deg(4) = 4$, $\deg(5) = 1$
- 6 is isolated; 5 is pendant

Degree

DEFINITION: Let $G = (V, E)$ be a directed graph. If $(u, v) \in E$, we say that u is **adjacent to** v and v is **adjacent from** u .

- u is the **initial vertex**_{起始点} of (u, v) ; v is the **terminal vertex**_{终点} of (u, v)
 - $u = v$: u is the initial vertex and the terminal vertex
- **in-degree**_{入度} $\deg^-(v)$: the number of edges where v is the terminal vertex
- **out-degree**_{出度} $\deg^+(v)$: the number of edges where v is the initial vertex
 - $u = v$: the loop contributes 1 to $\deg^-(v)$ and 1 to $\deg^+(v)$



- 5 is adjacent to 4; 4 is adjacent from 5
- 5 is the initial vertex of $(5, 4)$
- 4 is the terminal vertex of $(5, 4)$
- 1 is the initial and terminal vertex of a loop
- $\deg^-(1) = 2$; $\deg^+(1) = 2$
- $\deg^-(4) = 3$; $\deg^+(4) = 1$

Handshaking Theorem

THEOREM: Let $G = (V, E)$ be an undirected graph. Then
 $2|E| = \sum_{v \in V} \deg(v)$ and $|\{v \in V : \deg(v) \text{ is odd}\}|$ is even.

THEOREM: Let $G = (V, E)$ be a directed graph. Then
 $\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E|.$

Subgraph

DEFINITION: Let $G = (V, E)$ be a simple graph. $H = (W, F)$ is a **subgraph**_{子图} of G if $W \subseteq V$ and $F \subseteq E$.

- **proper subgraph**_{真子图}: H is a subgraph of G and $H \neq G$.
- The **subgraph induced**_{导出子图} by $W \subseteq V$ is (W, F) , where $F = \{e: e \in E, e \subseteq W\}$.
//Notation: $G[W]$
- The **subgraph induced**_{导出子图} by $F \subseteq E$ is (W, F) , where $W = \{v: v \in V, v \in e \text{ for some } e \in F\}$. //Notation: $G[F]$

DEFINITION:

Removing An Edge

Adding An Edge

Edge Contraction

Removing A Vertex

Complement

Graph Isomorphism

DEFINITION: The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are **isomorphic**_{同构} if there is a bijection $\sigma: V_1 \rightarrow V_2$ such that $\{u, v\} \in E_1 \Leftrightarrow \{\sigma(u), \sigma(v)\} \in E_2$.

- σ is called an **isomorphism**_{同构映射}
- **nonisomorphic:** not isomorphic

DEFINITION: Graph invariants are properties preserved by graph isomorphism. For example,

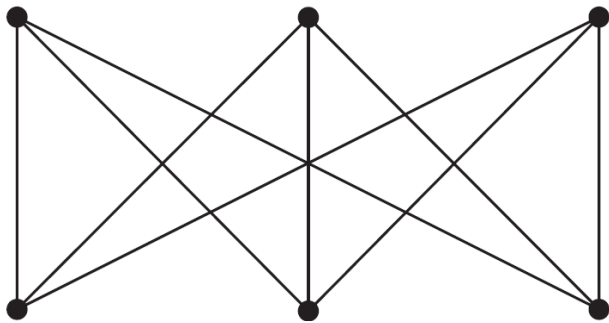
- The number of vertices
- The number of edges
- The number of vertices of each degree

REMARKS: The graph invariants can be used to determine if two graphs are isomorphic or not.

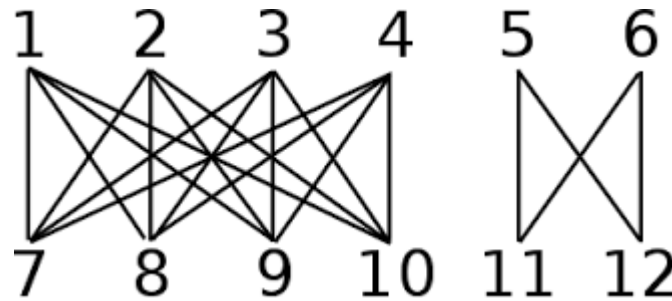
Bipartite Graph

DEFINITION: $G = (V, E)$ is a **bipartite graph** 二分图、二部图 if V has a partition $\{V_1, V_2\}$ such that $E \subseteq \{\{u_1, u_2\} : u_1 \in V_1, u_2 \in V_2\}$.

- (V_1, V_2) is a **bipartition** 二划分 of the vertex set V .



A bipartite graph of order 6



A bipartite graph of order 12

- $V_1 = \{1, 2, 3, 4, 5, 6\}$
- $V_2 = \{7, 8, 9, 10, 11, 12\}$

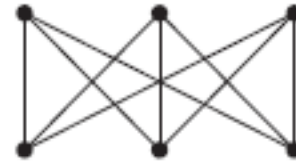
Complete Bipartite Graph

DEFINITION: A **complete bipartite graph** 完全二部图 is a graph $K_{m,n} = (V, E)$ with $V = \{x_1, \dots, x_m\} \cup \{y_1, \dots, y_n\}$ and $E = \{\{x_i, y_j\} : i \in [m], j \in [n]\}$

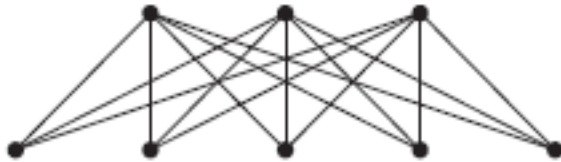
- Every vertex in V_1 is adjacent to every vertex in V_2



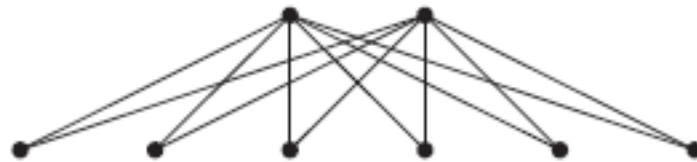
$K_{2,3}$



$K_{3,3}$



$K_{3,5}$



$K_{2,6}$

Bipartite Graph

Theorem

A simple graph is bipartite if and only if it is possible to assign one of two different colors to each vertex such that no two adjacent vertices have the same color.

Proof:

- If $G = (V, E)$ is bipartite, $V = V_1 \cup V_2$. Assign color c_1 to vertices of V_1 and color c_2 to vertices of V_2 .
- Reversely, suppose we can assign colors c_1 and c_2 to the vertices such that no two adjacent have the same. Let V_i be the set of vertices of color c_i , for $i = 1, 2$. Then $V = V_1 \cup V_2$. By assumption there are no edges connecting two vertices of V_1 or two vertices of V_2 , so each edge connects one vertex of V_1 with one vertex of V_2 . □

Matching

DEFINITION: Let $G = (V, E)$ be a simple graph. $M \subseteq E$ is a **matching**_{匹配} if $e \cap e' = \emptyset$ for every $e, e' \in M$. A vertex $v \in V$ is **matched** in M if $\exists e \in M$ such that $v \in e$, otherwise, v is **not matched**.

- **maximum matching**_{最大匹配}: a matching with largest number of edges.
- In a bipartite graph $G = (A \cup B, E)$, $M \subseteq E$ is a **complete matching**_{完全匹配} from A to B if every $u \in A$ is matched.

EXAMPLE: Marriage on an Island

- There are m boys $X = \{x_1, \dots, x_m\}$ and n girls $Y = \{y_1, \dots, y_n\}$
- $G = (X \cup Y, E = \{\{x_i, y_j\} : x_i \text{ and } y_j \text{ are willing to get married}\})$
- What is the largest number of couples that can be formed?

THEOREM (Hall 1935): A bipartite graph $G = (X \cup Y, E)$ has a complete matching from X to Y iff $|N(A)| \geq |A|$ for any $A \subseteq X$.

Path (Undirected)

- DEFINITION:** Let $G = (V, E)$ be an undirected graph and let $k \in \mathbb{N}$. A **path**_{路径} **of length k** from u to v in G is a sequence of k edges $e_1, \dots, e_k$ of G for which there exist vertices $x_0 = u, x_1, \dots, x_{k-1}, x_k = v$ such that $e_i = \{x_{i-1}, x_i\}$ for every $i \in [k]$.
- The path is **circuit**_{回路} if $u = v$ and $k > 0$
 - The path **passes through**_{经过} $x_1, \dots, x_{k-1}$
 - The path **traverses**_{遍历} $e_1, e_2, \dots, e_k$
 - The path is **simple**_{简单} if it doesn't contain an edge more than once.
 - If G is simple, the path can be denoted as $x_0, x_1, \dots, x_k$

Path (Directed)

DEFINITION: Let $G = (V, E)$ be a directed graph and let $k \in \mathbb{N}$. A **path of length k** from u to v in G is a sequence of k edges $e_1, \dots, e_k$ of G for which there exist vertices $x_0 = u, x_1, \dots, x_{k-1}, x_k = v$ such that $e_i = (x_{i-1}, x_i)$ for every $i \in [k]$.

- The path is a **circuit** if $u = v$ and $k > 0$
- The path **passes through** $x_1, \dots, x_{k-1}$
- The path **traverses** $e_1, e_2, \dots, e_k$
- The path is **simple** if it doesn't contain an edge more than once.
- If G has no multiple edges, the path can be denoted as $x_0, \dots, x_k$

Connectivity

DEFINITION: An undirected graph G is said to be **connected**_{连通的} if there is a path between any pair of distinct vertices.

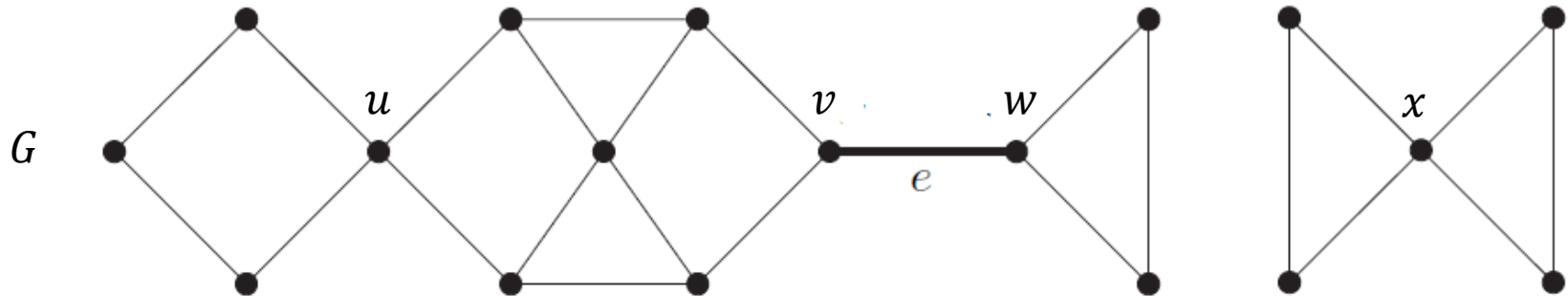
- Graph of order 1 is connected; the complete graph K_n is connected
- **disconnected**_{非连通的}: not connected
- **disconnect** G : remove vertices or edges to produce a disconnected subgraph

THEOREM: Let $G = (V, E)$ be a connected undirected graph. Then there is a simple path between any pair of distinct vertices.

Connected Component

DEFINITION: A **connected component**_{连通分支} of a graph $G = (V, E)$ is a connected subgraph of G that is not a proper subgraph of a connected subgraph of G . //i.e., maximal_{极大} connected subgraph

- $v \in V$ is a **cut vertex**_{割点} if $G - v$ has more connected components than G
- $e \in E$ is a **cut edge**_{割边}, **bridge**_桥 if $G - e$ has more connected components than G



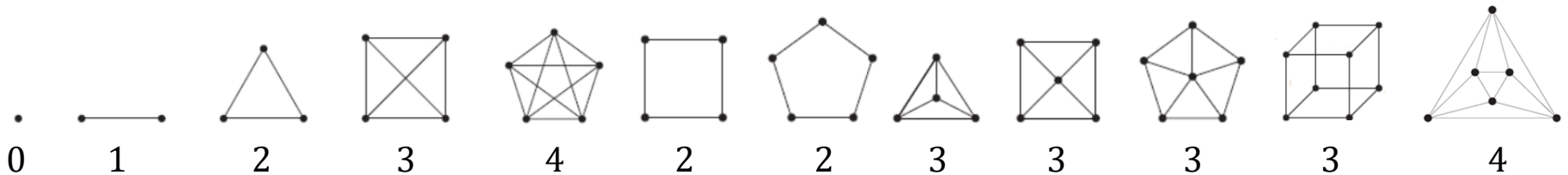
- There are 2 connected components in the graph G
- cut vertices: u, v, w, x
- cut edge: e

Vertex Connectivity

DEFINITION: A connected undirected graph $G = (V, E)$ is said to be **nonseparable**_{不可分的} if G has no cut vertex.

DEFINITION: Let $G = (V, E)$ be a connected simple graph.

- **vertex cut**_{点割集}: A subset $V' \subseteq V$ such that $G - V'$ is disconnected
- **vertex connectivity**_{点连通度} $\kappa(G)$: the minimum number of vertices whose removal disconnect G or results in K_1 ; equivalently,
 - if G is disconnected, $\kappa(G) = 0$; //additional definition
 - if $G = K_n$, $\kappa(G) = n - 1$ // K_n has no vertex cut
 - else, $\kappa(G)$ is the minimum size of a vertex cut of G



These graphs are all nonseparable

Vertex Connectivity

THEOREM: Let $G = (V, E)$ be a simple graph of order n . Then

- $0 \leq \kappa(G) \leq n - 1$
 - Removing $n - 1$ vertices gives K_1
 - $\kappa(G) \leq n - 1$
- $\kappa(G) = 0$ iff G is disconnected or $G = K_1$
 - trivial
- $\kappa(G) = n - 1$ iff $G = K_n (n \geq 2)$
 - If: obvious
 - Only if:
 - $n = 2: \kappa(G) = 1 \Rightarrow G = K_2$
 - $n \geq 3$: Prove by contradiction. Suppose that $G \neq K_n$.
 - There exist distinct $u, v \in V$ such that $u \neq v$ and $\{u, v\} \notin E$
 - Let $X = V - \{u, v\}$. Then $G - X$ is disconnected.
 - $\kappa(G) \leq |X| = n - 2 < n - 1$.

Vertex Connectivity

DEFINITION: A simple graph $G = (V, E)$ is called **k -connected** _{k 点连通的} (**k -vertex-connected**) _{k 点连通的} if $\kappa(G) \geq k$.

THEOREM: Let $G = (V, E)$ be a simple graph of order n . Then

- G is 1-connected iff G is connected and $G \neq K_1$.
 - **Only if:** G disconnected or $G = K_1 \Rightarrow \kappa(G) = 0$
 - **If:** $G \neq K_1 \Rightarrow n \geq 2$; G is connected $\Rightarrow$ removing 0 vertex cannot disconnect G or give $K_1 \Rightarrow \kappa(G) \geq 1$
- G is 2-connected iff G is nonseparable and $n \geq 3$.
 - **Only if:** $n \leq 2 \Rightarrow \kappa(G) \leq 1$; G not nonseparable $\Rightarrow G$ has cut vertex $\Rightarrow \kappa(G) \leq 1$.
 - **If:** $n \geq 3 \Rightarrow$ removing ≤ 1 vertex cannot result in K_1 ; G nonseparable $\Rightarrow$ removing ≤ 1 vertex cannot disconnect G ; Hence. $\kappa(G) \geq 2$.
- G is k -connected iff G is j -connected for all $j \in \{0, 1, \dots, k\}$
 - **Only if:** $\kappa(G) \geq k \Rightarrow \kappa(G) \geq j$ for all $j \in \{0, 1, \dots, k\} \Rightarrow G$ is j connected
 - **If:** G is obviously k -connected

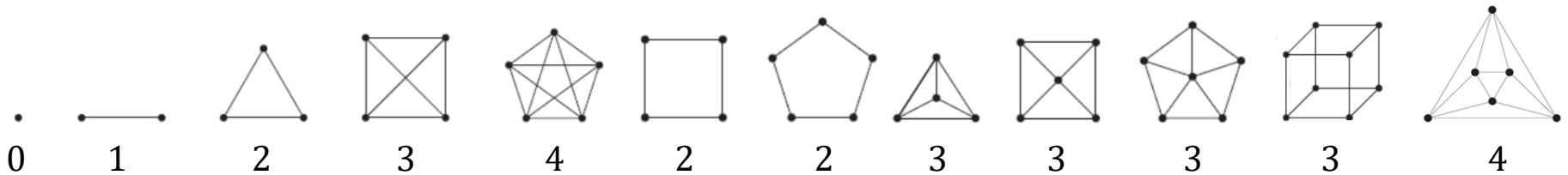
Edge Connectivity

DEFINITION: Let $G = (V, E)$ be a connected simple graph. $E' \subseteq E$ is an **edge cut**_{边割集} of G if $G - E'$ is disconnected.

DEFINITION: Let $G = (V, E)$ be a simple graph.

The **edge connectivity**_{边连通度} ($\lambda(G)$) of G is defined as below:

- G disconnected: $\lambda(G) = 0$
- G connected:
 - $|V| = 1$: $\lambda(G) = 0$
 - $|V| > 1$: $\lambda(G)$ is the minimum size of edge cuts of G .



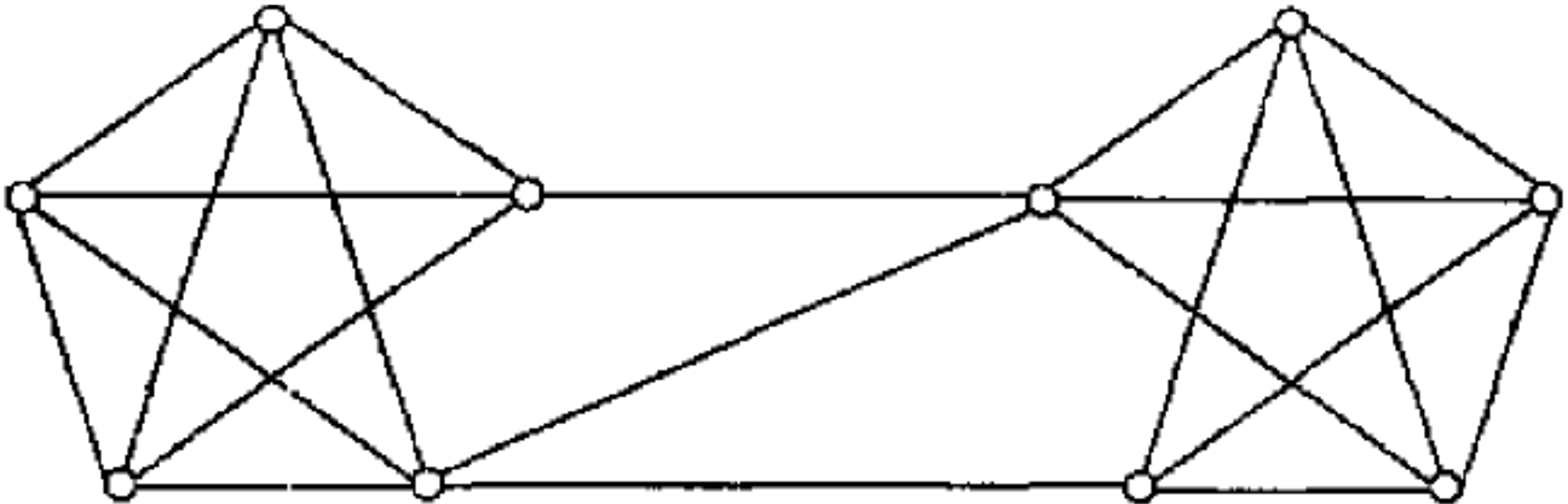
Edge Connectivity

THEOREM: Let $G = (V, E)$ be a simple graph of order n . Then

- $0 \leq \lambda(G) \leq n - 1$
 - $n = 1$: $G = K_1$ and $\lambda(G) = 0$
 - $n > 1$: $\deg(u) \leq n - 1$ for every $u \in V$
 - By removing $\{\{u, x\} : \{u, x\} \in E\}$, we can disconnect G .
 - Hence, $\lambda(G) \leq n - 1$.
- $\lambda(G) = 0$ iff G is disconnected or $G = K_1$
 - Only if: $n > 1$ and G connected $\Rightarrow \lambda(G) \geq 1$;
 - If: definition
- $\lambda(G) = n - 1$ iff $G = K_n$ ($n \geq 2$)
 - Only if: if $G \neq K_n$, then $\deg(u) < n - 1$ for some $u \in V$.
 - Remove $\{\{u, x\} : \{u, x\} \in E\}$. Then G is disconnected. $\lambda(G) < n - 1$
 - If: $\lambda(K_n) \geq \kappa(K_n) = n - 1$. (see the next theorem)

Connectivity

THEOREM: Let $G = (V, E)$ be a simple graph. Then $\kappa(G) \leq \lambda(G) \leq \delta(G)$, where $\delta(G) = \min_{v \in V} \deg(v)$ is the least degree of G 's vertices.



- $\kappa(G) = 2$
- $\lambda(G) = 3$
- $\delta(G) = 4$

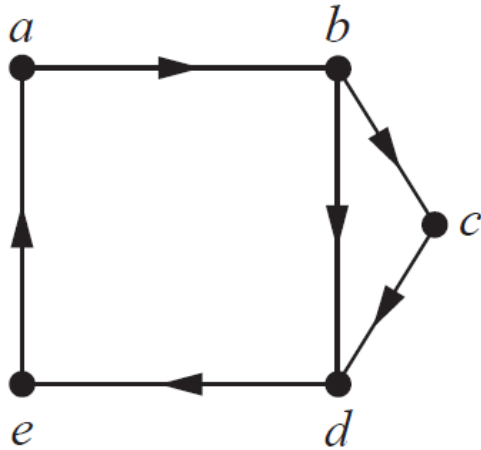
https://cp-algorithms.com/graph/edge_vertex_connectivity.html

<http://www.math.caltech.edu/~2014-15/2term/ma006b/05%20connectivity%201.pdf>

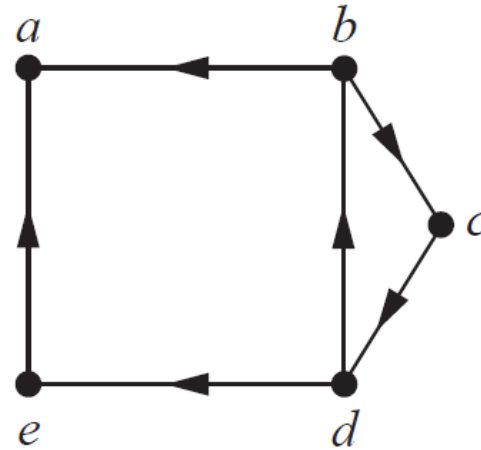
Connected Directed Graphs

DEFINITION: Let $G = (V, E)$ be a directed graph. G is said to be **strongly connected** if there is a path from u to v and a path from v to u for all $u, v \in V$ ($u \neq v$).

- **weakly connected:** the graph is connected if we remove the directions of all direct edges.



Strongly connected



Weakly connected

Paths and Isomorphism

Theorem

The existence of a simple circuit of length k , $k \geq 3$ is an isomorphism invariant for simple graphs.

Counting Paths Between Vertices

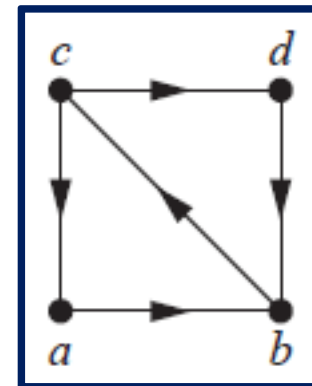
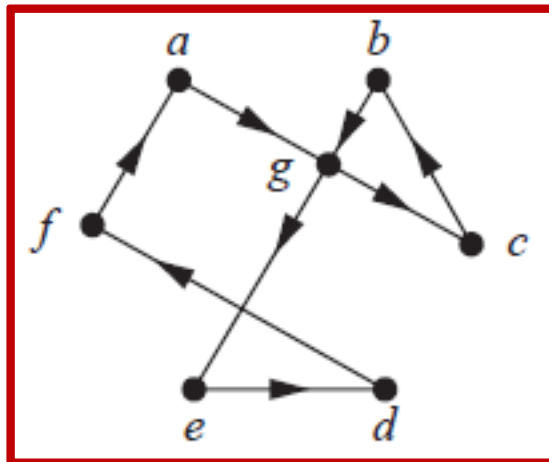
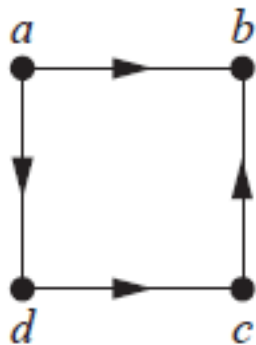
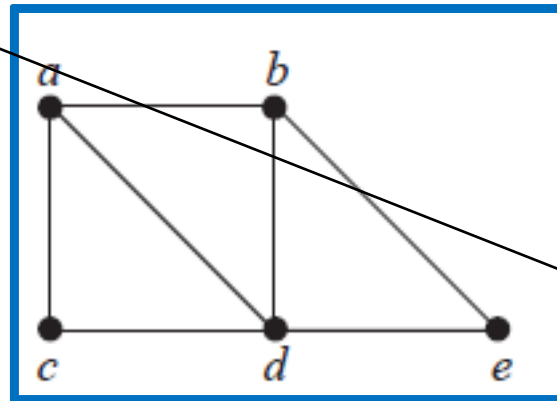
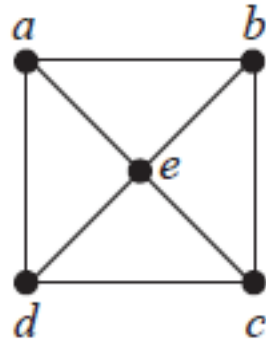
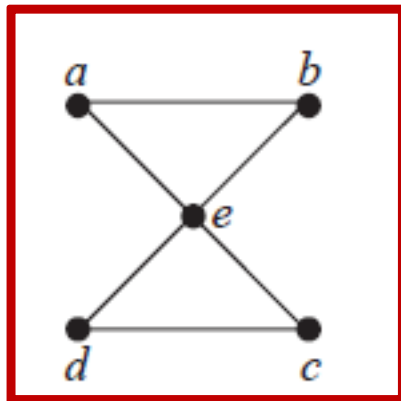
Theorem

Let G be a graph with adjacency matrix A with respect to the ordering of vertices $v_1, \dots, v_n$. The number of different paths of length $r \geq 1$ from v_i to v_j equals the (i, j) entry of the matrix A^r .

Euler Paths and Circuits

DEFINITION: Let $G = (V, E)$ be a graph.

- **Euler Path** 欧拉路径: a simple path that traverses every edge of G .
- **Euler Circuit** 欧拉回路: a simple circuit that traverses every edge of G .



Remark: When G has multiple edges, these edges will be given different names and considered as different. This is implicit in the textbook.

Euler Circuits

THEOREM: Let $G = (V, E)$ be a connected multigraph of order ≥ 2 .
Then G has an Euler circuit iff $2 \mid \deg(x)$ for every $x \in V$.

ALGORITHM (Hierholzer):

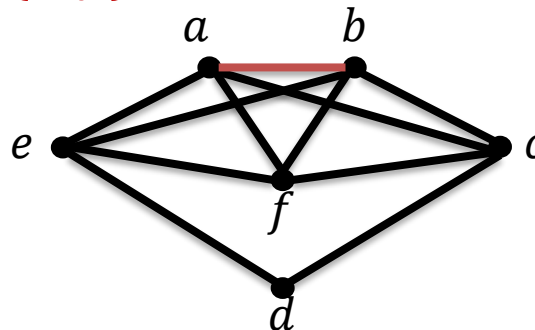
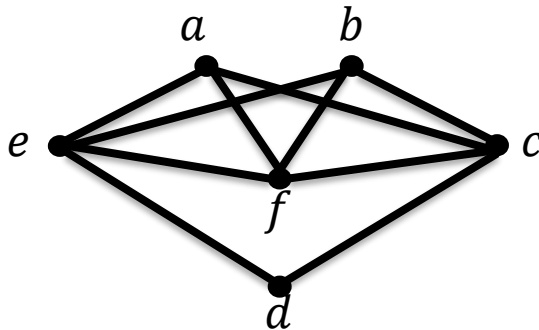
- **Input:** $G = (V, E)$, a connected multigraph, $2 \mid \deg(x), \forall x \in V$
- **Output:** an Euler circuit
 - **circuit:** = a circuit in G
 - $H := G - \mathbf{circuit}$ – isolated vertices
 - while H has edges do
 - **subcircuit:** = a circuit in H that intersects **circuit**
 - $H := H - \mathbf{subcircuit}$ – isolated vertices
 - **circuit:** = **circuit** $\cup$ **subcircuit**
 - return **circuit**

Euler Paths

THEOREM: Let $G = (V, E)$ be a connected multigraph of order ≥ 2 . Then G has an Euler path (not Euler circuit) iff G has exactly 2 vertices of odd degree.

ALGORITHM:

- **Input:** $G = (V, E)$, a connected multigraph, $x, y \in V$ have odd degrees
- **Output:** an Euler path
 - $H := G + \{x, y\}$
 - find an Euler circuit using Hierholzer's algorithm
 - remove the edge $\{x, y\}$ from the circuit

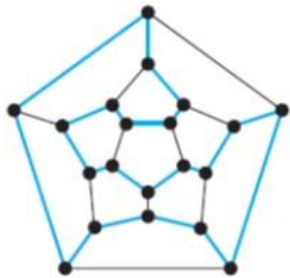


$a, c, d, e, f, b, a, e, b, c, f, a$
 $a, c, d, e, f, \textcolor{red}{b}, \textcolor{red}{a}, e, b, c, f, a$
 $\textcolor{red}{a}, e, b, c, f, a, c, d, e, f, \textcolor{red}{b}$

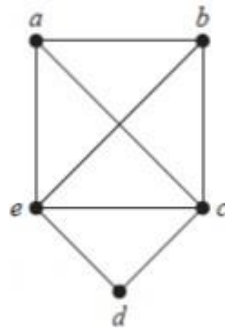
Hamilton Paths and Circuits

DEFINITION: Let $G = (V, E)$ be a graph.

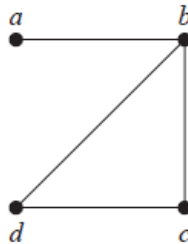
- **Hamilton Path:** A simple path that passes through every vertex exactly once.
- **Hamilton Circuit:** A simple circuit that passes through every vertex exactly once.



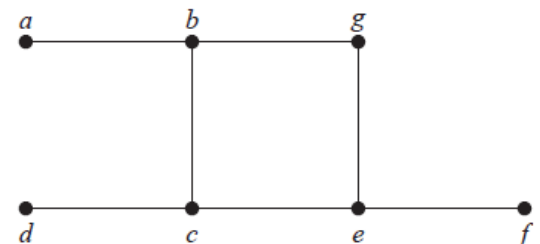
✓ Hamilton path
✓ Hamilton circuit



✓ Hamilton path
✓ Hamilton circuit



✓ Hamilton path
× Hamilton circuit



× Hamilton path
× Hamilton circuit

Hamilton Circuits

Determine if there is a Hamilton circuit in a given graph G ?

- This problem is NP-Complete. //that means very difficult

Necessary conditions on Hamilton circuit.

- If G has a vertex of degree 1, then G cannot have a Hamilton circuit.
- If G has a vertex of degree 2, then a Hamilton circuit of G traverses both edges.

Sufficient conditions on Hamilton circuit.

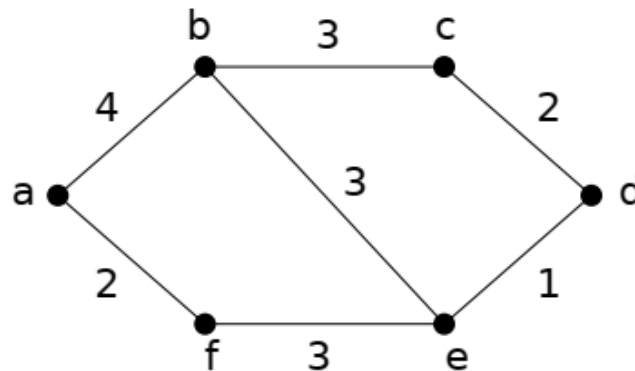
- **Ore's Theorem:** Let $G = (V, E)$ be a simple graph of order $n \geq 3$. If $\deg(u) + \deg(v) \geq n$ for all $\{u, v\} \notin E$, then G has a Hamilton circuit.
- **Dirac's Theorem:** Let $G = (V, E)$ be a simple graph of order $n \geq 3$. If $\deg(u) \geq n/2$ for every $u \in V$, then G has a Hamilton circuit.
 - This is a corollary of Ore's Theorem
 - $\forall u \in V, \deg(u) \geq n/2 \Rightarrow \forall u, v \in V, \deg(u) + \deg(v) \geq n$

Shortest Path Problem

Definition

A **weighted graph** is a graph $G = (V, E)$ such that each edge is assigned with a strictly positive number.

The **length** of a path in weighted graph is the sum of the weights of the edges of this path.



a, b, c is a path of length 7 and b, e, d, c is a path of length 6

Remark: Observe that in a non-weighted graph the length of a path is the number of edges in the path!

Dijkstra's Algorithm

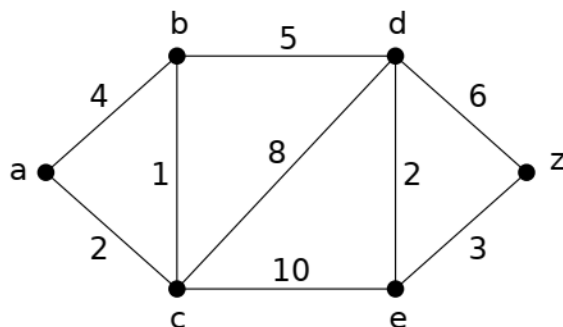
Notations: S_k := distinguished set after k iterations, $L_k(v)$:= length of a shortest path from a to v containing only vertices in S_k ("label" of v).

Initialization: $L_0(a) = 0$,
 $L_0(v) = \infty$ for every vertex $v \neq a$,
 $S_0 = \emptyset$.

k th iteration:

- S_k is formed from S_{k-1} by adding a vertex u not in S_{k-1} with smallest label,
- Update the labels of all vertices not in S_k so that $L_k(v)$ is the length of a shortest path from a to v containing only vertices in S_k , i.e.

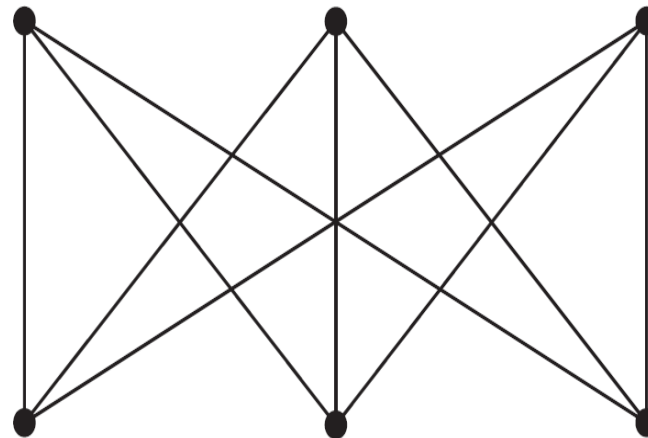
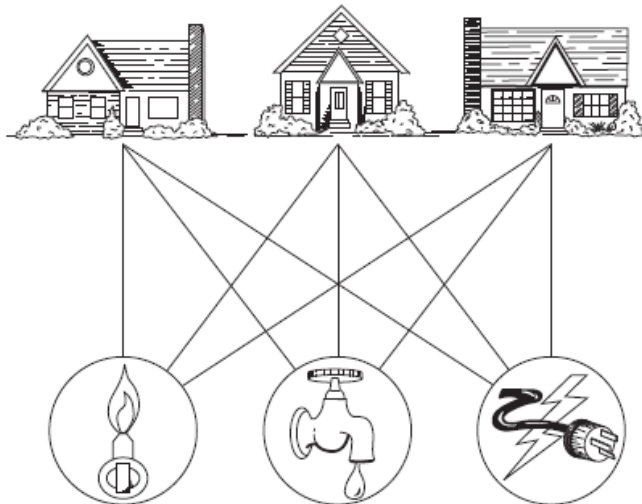
$L_k(v) = \min\{L_{k-1}(v), L_{k-1}(u) + w(u, v)\}$ (with $w(u, v)$ length of the edge (u, v))



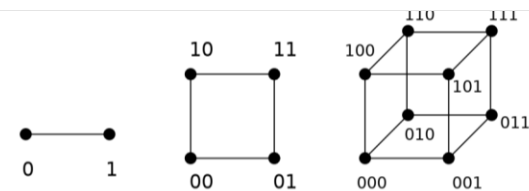
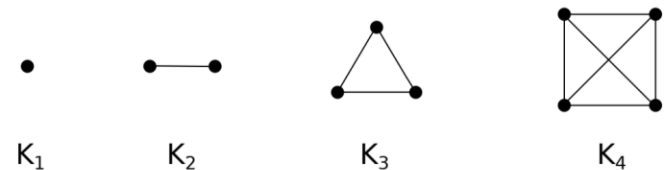
Planar Graph

DEFINITION: Let $G = (V, E)$ be an undirected graph. G is called a **planar graph** 平面图 if it can be drawn in the plane without any edges crossing.

- Crossing of edges: an intersection other than endpoints (vertices)
- **planar representation** 平面表示: a drawing w/o edge crossing; **nonplanar** 非平面的



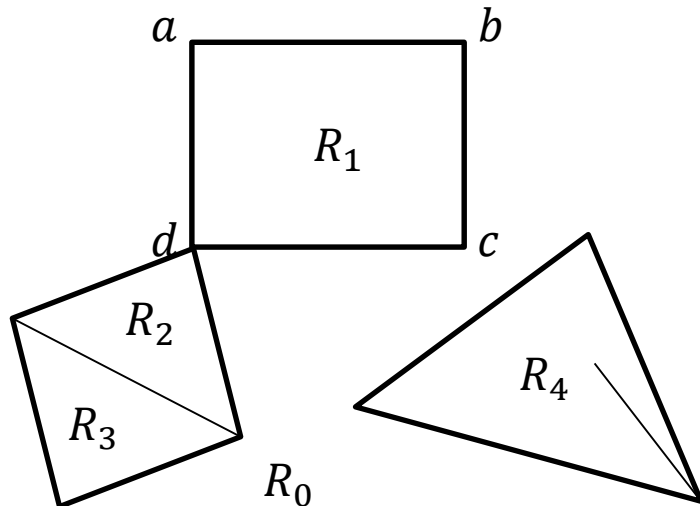
- K_1, K_2, K_3, K_4 are planar graphs
- $K_{1,n}, K_{2,n}$ are planar graphs
- C_n ($n \geq 3$), W_n ($n \geq 3$) are planar graphs
- Q_1, Q_2, Q_3 are planar graphs



Regions

DEFINITION: Let $G = (V, E)$ be a planar graph. Then the plane is divided into several **regions**面 by the edges of G .

- The infinite region is **exterior region**外部面. The others are **interior regions**内部面.
- The **boundary**边界 of a region is a subset of E .
- The **degree**度数 of a region is the number of edges on its boundary.
 - If an edge is shared by R_i, R_j , then it contributes 1 to $\deg(R_i), \deg(R_j)$
 - If an edge is on the boundary of a single region R_i , then it contributes 2 to $\deg(R_i)$



- The plane is divided into 5 regions R_0, R_1, R_2, R_3, R_4
 - R_0 is the exterior region
 - R_1, R_2, R_3, R_4 are interior regions
- The boundary of R_1 ; $\deg(R_1) = 4$
- There are 4 edges on the boundary of R_4
 - $\deg(R_4) = 1 + 1 + 1 + 2 = 5$ because one of the edges contribute 2 to $\deg(R_4)$
- $\deg(R_0) = 11, \deg(R_1) = 4, \deg(R_2) = 3, \deg(R_3) = 3, \deg(R_4) = 5$

Euler's Formula

THEOREM: Let $G = (V, E)$ be a connected planar simple graph with e edges and v vertices. Let r be the number of regions in a planar representation of G . Then $r = e - v + 2$.

THEOREM: Let G be a planar simple graph with p connected components. Then $|V(G)| - |E(G)| + |R(G)| = p + 1$.

- Let $G_1, G_2, \dots, G_p$ be the connected components of G .
 - By Euler's formula, $|R(G_i)| = |E(G_i)| - |V(G_i)| + 2$ for all $i \in [p]$
- $|V(G)| = |V(G_1)| + |V(G_2)| + \dots + |V(G_p)|$
- $|E(G)| = |E(G_1)| + |E(G_2)| + \dots + |E(G_p)|$
- $|R(G)| = |R(G_1)| + |R(G_2)| + \dots + |R(G_p)| - p + 1$
- $|V(G)| - |E(G)| + |R(G)| = \sum_{i=1}^p (|V(G_i)| - |E(G_i)| + |R(G_i)|) - p + 1$
 $= 2p - p + 1 = p + 1$

Application

THEOREM: Let G be a connected planar simple graph. If every region has degree $\geq l$ in a planar representation of G , then

then $|E(G)| \leq \frac{l}{l-2} (|V(G)| - 2)$.

- Let $R_1, \dots, R_t$ be the regions given by a planar representation of G // $t = |R(G)|$
 - $\deg(R_i) \geq l$ for every $i = 1, 2, \dots, t$
- Let $r = \deg(R_1) + \deg(R_2) + \dots + \deg(R_t)$. Then $r = 2|E(G)|$.
 - Every edge contributes 2 to r
 - If $e \in E$ is on the boundary of a single region R_i , then e contributes 2 to $\deg(R_i)$;
 - If $e \in E$ is shared by R_i and R_j , then e contributes 1 to $\deg(R_i)$ and 1 to $\deg(R_j)$;
- $2|E(G)| = r = \deg(R_1) + \deg(R_2) + \dots + \deg(R_t) \geq lt = l|R(G)|$
- $|R(G)| = |E(G)| - |V(G)| + 2$
- Hence, $|E(G)| \leq \frac{l}{l-2} (|V(G)| - 2)$

Application

COROLLARY: Let G be a connected planar simple graph. If $|V(G)| \geq 3$, then $|E(G)| \leq 3|V(G)| - 6$.

- Every region has degree ≥ 3 in a planar representation of G
- Let $l = 3$ in the previous theorem
 - $|E(G)| \leq \frac{3}{3-2} (|V(G)| - 2) = 3|V(G)| - 6$.

EXAMPLE: The complete graph K_5 is not planar.

- $|E(K_5)| = \binom{5}{2} = 10, |V(K_5)| = 5, K_5$ is connected simple and of order ≥ 3
- $|E(K_5)| > 3|V(K_5)| - 6$
 - Hence, K_5 cannot be planar

COROLLARY: Let G be a connected planar simple graph. Then G has a vertex of degree ≤ 5 .

- $|V(G)| < 3$: the statement is true.
- $|V(G)| \geq 3: \forall u \in V(G), \deg(u) \geq 6 \Rightarrow 2|E(G)| = \sum_u \deg(u) \geq 6|V(G)|$
 - G cannot be planar

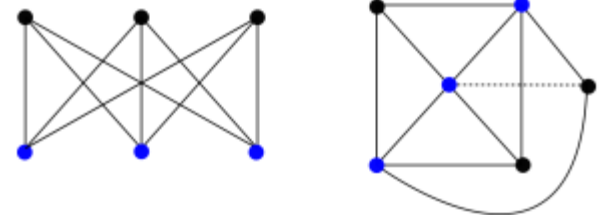
Application

COROLLARY: Let G be a connected planar simple graph. If $|V(G)| \geq 3$ and there is no circuits of length 3 in G , then $|E(G)| \leq 2|V(G)| - 4$.

- Let $R_1, \dots, R_t$ be the regions given by a planar representation of G // $t = |R(G)|$
 - $\deg(R_i) \geq 4$ for every $i = 1, 2, \dots, t$
- Hence, $|E(G)| \leq \frac{4}{4-2} (|V(G)| - 2) = 2|V(G)| - 4$

EXAMPLE: The complete bipartite graph $K_{3,3}$ is not planar.

- $|E(K_{3,3})| = 3 \times 3 = 9, |V(K_{3,3})| = 3 + 3 = 6 \geq 3$
- $K_{3,3}$ is connected, simple and of order ≥ 3 .
- There is no circuits of length 3 in $K_{3,3}$
- $|E(K_{3,3})| = 9 > 8 = 2|V(K_{3,3})| - 4$
- Hence, $K_{3,3}$ cannot be planar

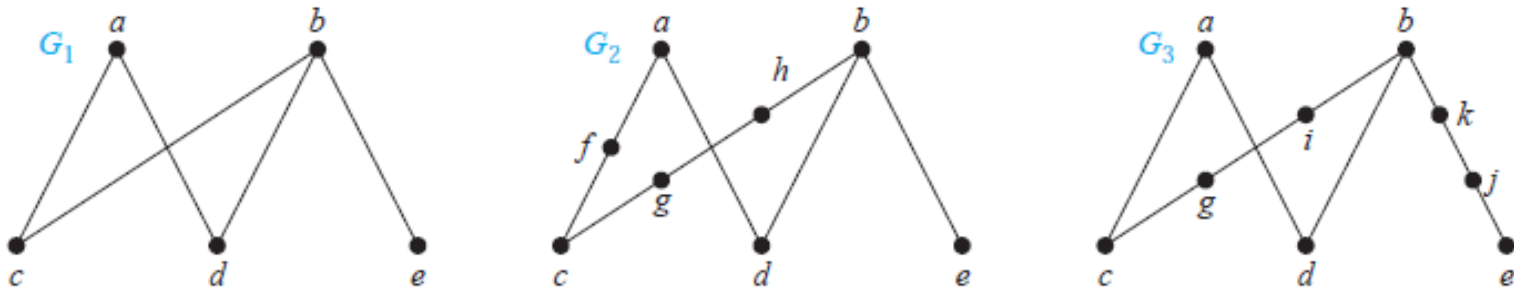


REMARKS: K_5 and $K_{3,3}$ are fundamental nonplanar graphs.

Homeomorphic

DEFINITION: Let $G = (V, E)$ be a graph and $\{u, v\} \in E$.

- **elementary subdivision** 初等细分: $G' = (V \cup \{w\}, E - \{u, v\} + \{u, w\} + \{v, w\})$
- Two graphs are **homeomorphic** 同胚的 if they can be obtained from the same graph via elementary subdivisions

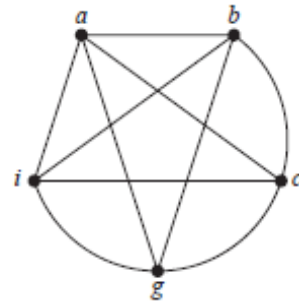
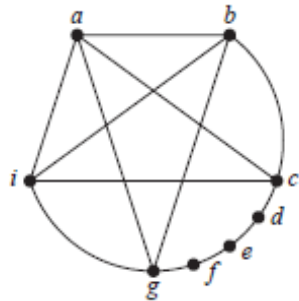
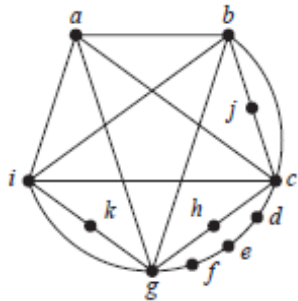


G_2 and G_3 are homeomorphic

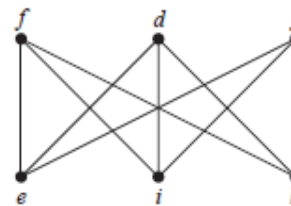
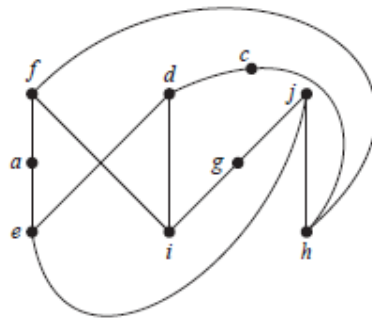
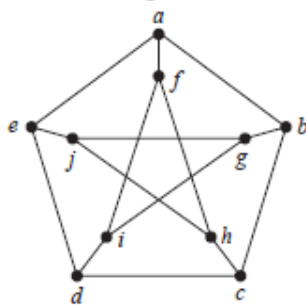
Kuratowski's Theorem

THEOREM: A graph G is nonplanar if and only if it has a subgraph homeomorphic to $K_{3,3}$ or K_5 .

EXAMPLE: The following graph is nonplanar.



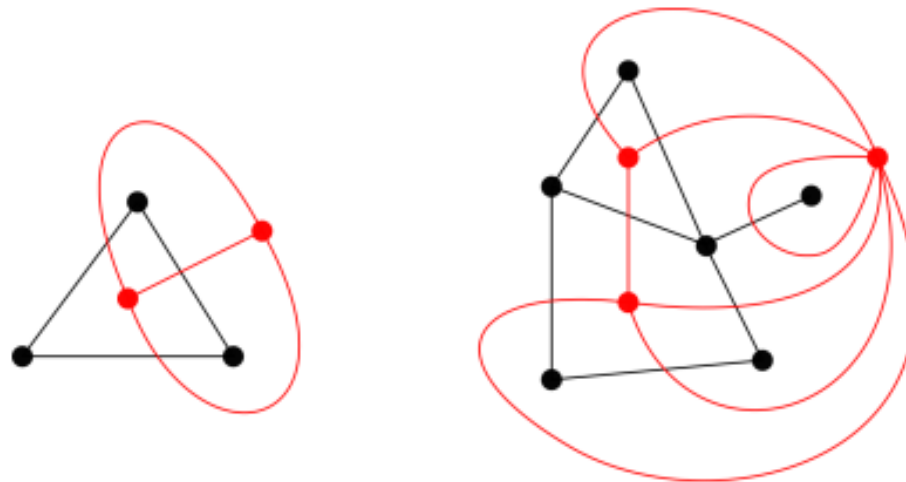
There is a subgraph
homeomorphic to K_5



There is a subgraph
homeomorphic to $K_{3,3}$

Dual Graph

Let G be a planar graph and assume we take a planar representation of G that we denote also G . The **dual of G** is the graph G^* that has a vertex for each face of G and an edge connecting two vertices if the corresponding faces in G have a common edge in their boundary.

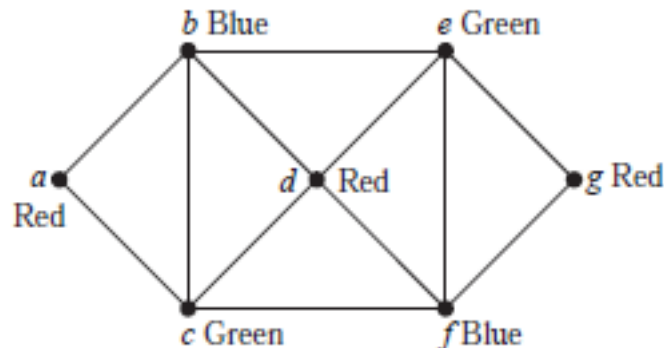
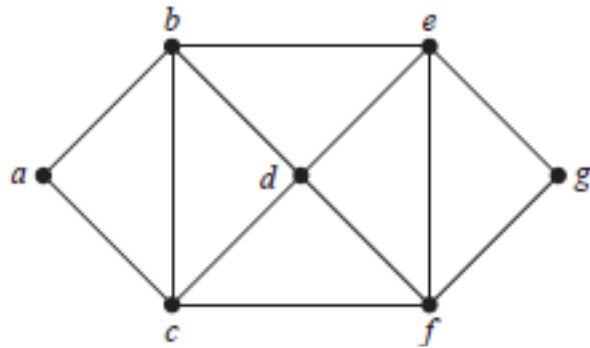


Remark: The dual of a planar simple graph is not necessarily simple.

Graph Coloring

DEFINITION: Let $G = (V, E)$ be a simple graph. A **k -coloring** _{k -着色} of G is a map $f: V \rightarrow [k]$ such that $f(u) \neq f(v)$ whenever $\{u, v\} \in E$.

- **chromatic number** ($\chi(G)$)_{色数}: the least k s.t. G has a k -coloring.



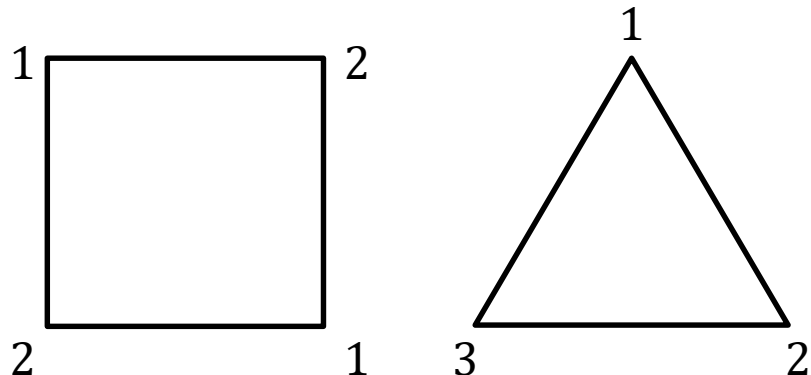
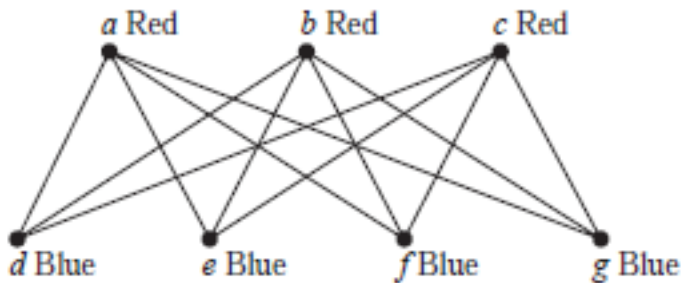
$$\chi(G) = 3$$

The chromatic number is at least 3 because $a; b; c$ is a circuit of length 3

Graph Coloring

THEOREM: Let $G = (V, E)$ be a simple graph.

- $1 \leq \chi(G) \leq |V|$
- $\chi(G) = 1$ iff $E = \emptyset$
- $\chi(G) = 2$ iff G is bipartite and $|E| \geq 1$.
- $\chi(K_n) = n$ for every integer $n \geq 1$.
 - $\chi(G) \geq n$ if G has a subgraph isomorphic to K_n
- $\chi(C_n) = 2$ if $2|n$; $\chi(C_n) = 3$ if $2 \nmid (n - 1)$; ($n \geq 3$)
- $\chi(G) \leq \Delta(G) + 1$, where $\Delta(G) = \max\{\deg(v) : v \in V\}$.



4/5-coloring Theorem

Theorem (Four coloring Theorem)

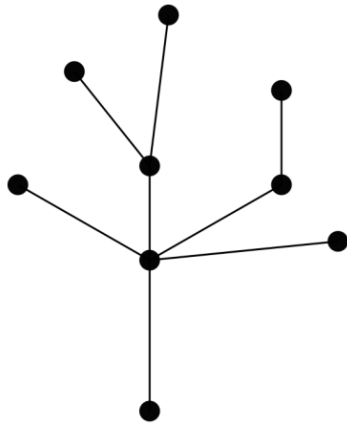
The chromatic number of a simple planar graph is no greater than 4.

Remarks: The proof of the 4-coloring Theorem depends on a computer. The two previous theorems are true for planar graphs only. A non planar graph can have an arbitrarily large chromatic number.

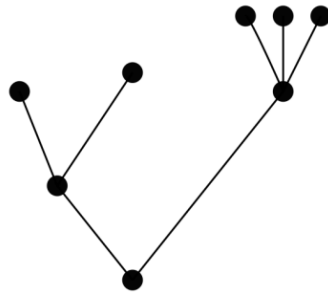
Tree

Definition

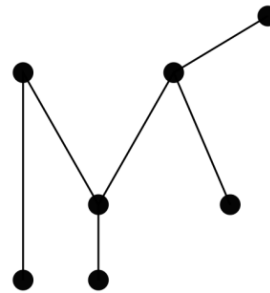
- A **tree** is a connected undirected graph with no simple circuits.
- A **forest** is an graph such that each of its connected components is a tree.



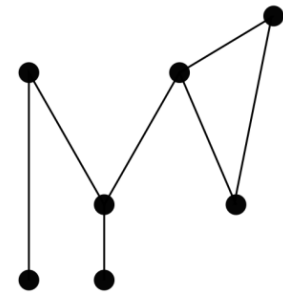
G



H



I



K

G , H , I are trees, but K is not a tree.

Properties of Tree

Tree = connected with no simple circuit (definition)

- (1) connected
- (2) no simple circuit
- (3) $(n - 1)$ edges (n =nb of vertices)

Previous theorem: $(1) + (2) \Rightarrow (3)$

We also have: $(1) + (3) \Rightarrow (2)$
 $(2) + (3) \Rightarrow (1)$

Example: For what value of m, n the complete bipartite graph $K_{m,n}$ is a tree?

$K_{m,n}$ is connected, has $m + n$ vertices and $m \times n$ edges.

It is a tree if:

$$m \times n = m + n - 1 \iff (n - 1)m = n - 1$$

If $n \neq 1$: $m = 1$

If $n = 1$: $m \in \mathbb{N}^*$

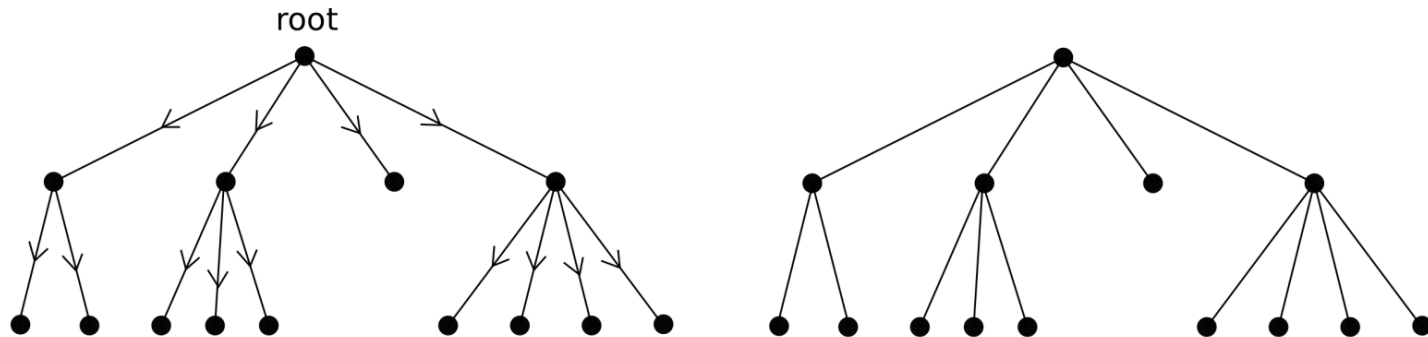
Rooted Tree

Definition

A **rooted tree** is a tree in which one vertex has been designated as the root and every edge is directed away from the root.

Remarks: • A rooted tree is a directed graph.

- We usually draw a rooted tree with its root at the top of the graph.
- We usually omit the arrows on the edges to indicate the direction because it is uniquely determined by the choice of the root.
- Any non rooted tree can be changed to a rooted tree by choosing a vertex for the root.

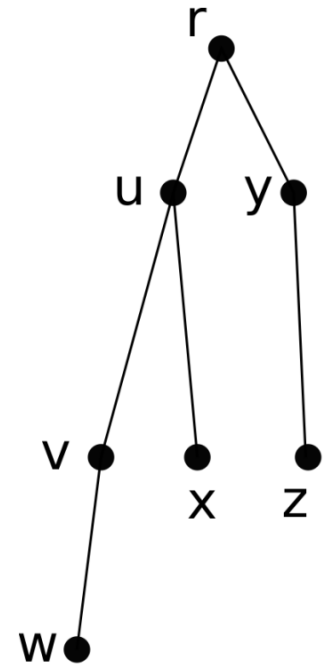


Rooted Tree

Definition

Let T be a rooted tree and v a vertex which is not the root. We call

- **parent** of v the *unique* vertex u such that there is an edge from u to v ,
- **child** of v a vertex w such that there is an edge from v to w ,
- **siblings** vertices with the same parent,
- **ancestors** of v all vertices in the path from the root to v ,
- **descendants** of v all vertices that have v as an ancestor,
- **leaf** a vertex which has no children,
- **internal vertex** a vertex that has children,
- **subtree with v at its root** the subgraph of T consisting of v and its descendants and the edges incident to them.

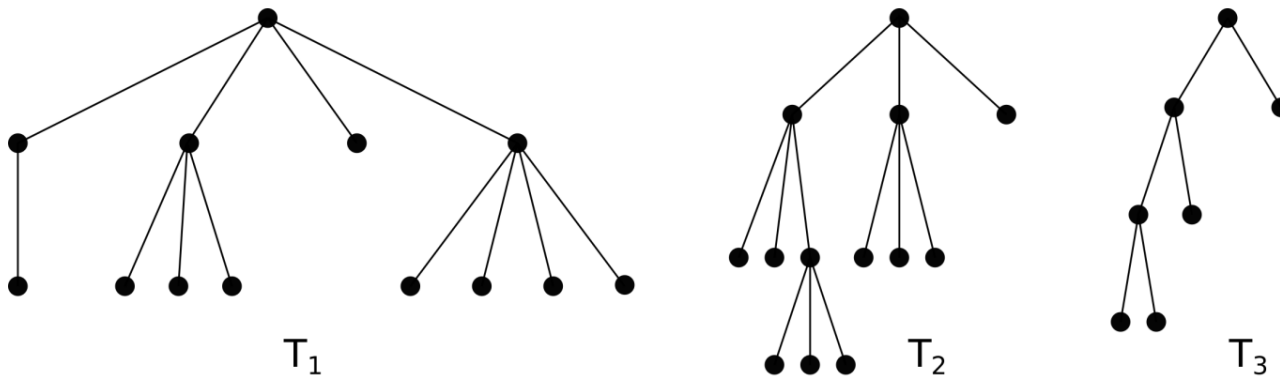


- r is the root
- v is child of u and parent of w
- v and x are siblings

Rooted Tree

Definition

- A rooted tree is called an ***m*-ary tree** if every internal vertex has no more than m children.
- A rooted tree is called a **full *m*-ary tree** if every internal vertex has exactly m children.
- An m -ary tree with $m = 2$ is called a **binary tree**. In this case if an internal vertex has two children, they are called **left child** and **right child**. The subtree rooted at the left (resp. right) child of a vertex is called the **left (resp. right) subtree** of this vertex.



T_1 is a 4-ary tree, T_2 a full 3-ary tree, T_3 a full binary tree.

Properties of Tree

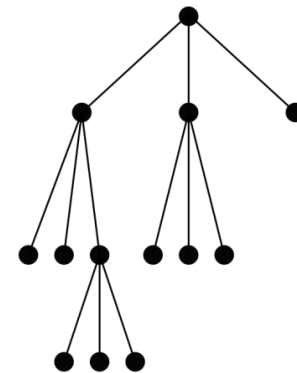
Theorem

A full m -ary tree with i internal vertices contains $n = mi + 1$ vertices.

Proof: Each vertex (except the root) is the child of an internal vertex.

There are i internal vertices, each with m children

$\Rightarrow mi$ vertices + root = $mi + 1$ vertices



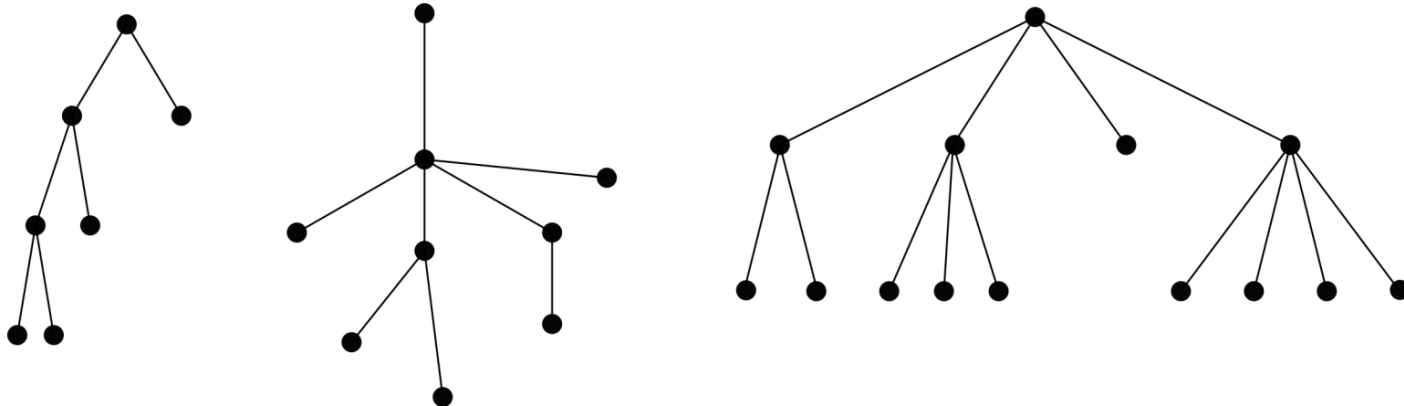
A full m -ary tree with

- 1** n vertices has $i = (n - 1)/m$ internal vertices and $\ell = ((m - 1)n + 1)/m$ leaves,
- 2** i internal vertices has $n = mi + 1$ vertices and $\ell = (m - 1)i + 1$ leaves,
- 3** ℓ leaves has $n = (m\ell - 1)/(m - 1)$ vertices and $i = (\ell - 1)/(m - 1)$ internal vertices.

Balanced m-ary Tree

Definition

- The **level** of a vertex v in a rooted tree is the length of the unique path from the root to this vertex.
- The **height** of a rooted tree is the maximum of the levels of its vertices.
- A rooted m -ary tree of height h is **balanced** if all leaves are at levels h or $h - 1$.



Balanced m-ary Tree

Theorem

There are at most m^h leaves in an m-ary tree of height h .

Proof: Induction again!

Corollary

If an m-ary tree of height h has l leaves, then $h \geq \lceil \log_m l \rceil$. If moreover the m-ary tree is full and balanced, then $h = \lceil \log_m l \rceil$.

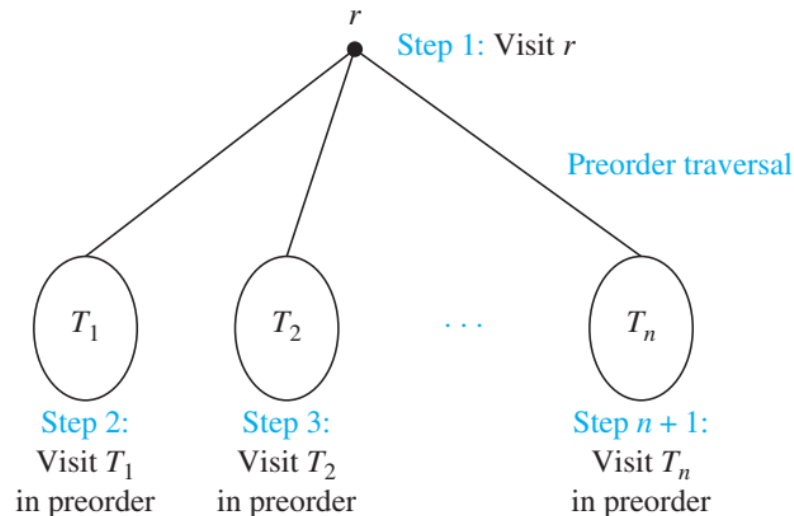
Tree Traversals

Preorder traversal algorithm

Recursive definition: Let T be a rooted tree with root r

- if T consists only on r : r is the preorder traversal of T .
- otherwise, denote by $T_1, \dots, T_n$ the subtrees rooted at the children of r , from left to right.

The preorder traversal of T begins by visiting r , then traverses T_1 in preorder, then T_2 in preorder, ..., and finally T_n in preorder.



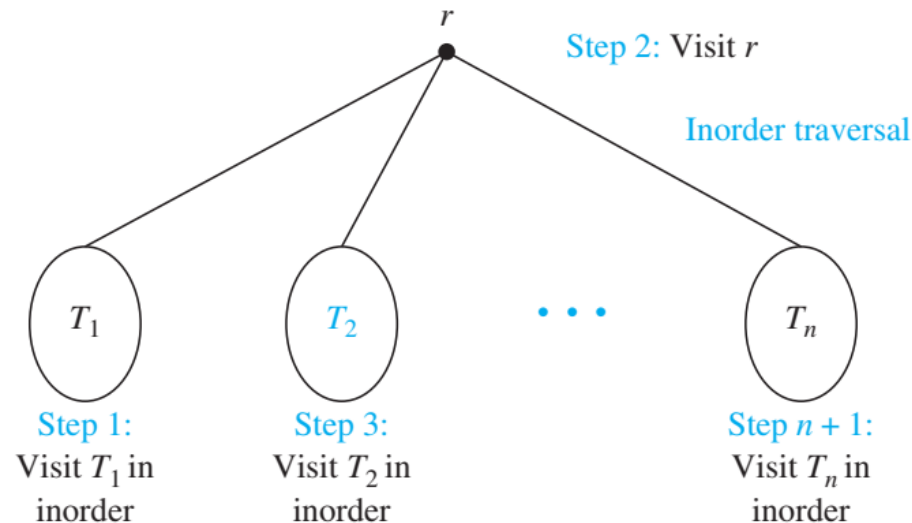
Tree Traversals

Inorder traversal algorithm

Recursive definition: Let T be a rooted tree with root r

- if T consists only on r : r is the inorder traversal of T .
- otherwise, denote by $T_1, \dots, T_n$ the subtrees rooted at the children of r , from left to right.

The inorder traversal of T begins by traversing T_1 in inorder, then visiting r , then traversing T_2 in inorder, then T_3 in inorder, ..., and finally T_n in inorder.



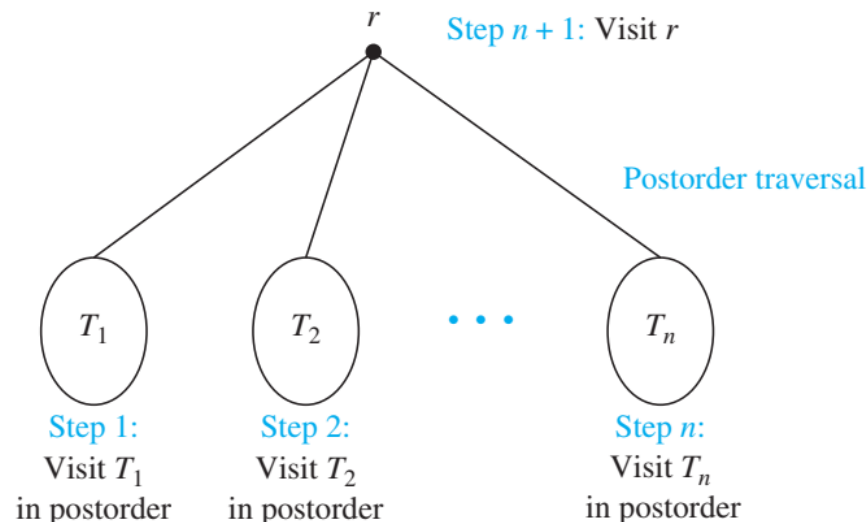
Tree Traversals

Postorder traversal algorithm

Recursive definition: Let T be a rooted tree with root r

- if T consists only on r : r is the postorder traversal of T .
- otherwise, denote by $T_1, \dots, T_n$ the subtrees rooted at the children of r , from left to right.

The postorder traversal of T begins by traversing T_1 in postorder, then T_2 in postorder, ..., then T_n in postorder, and ends by visiting the root r .

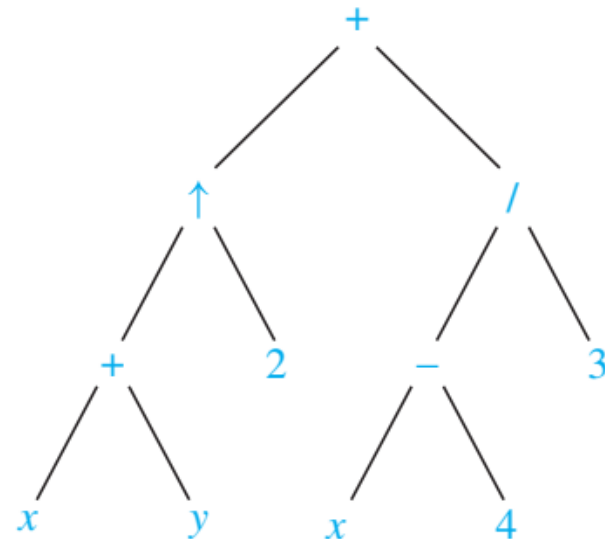


Infix, Prefix, Postfix Notation

Goal: Using ordered rooted trees to represent arithmetic expressions or compound propositions.

- leaves: numbers or variables,
- internal vertices: operations, where each operation operates on its left and right subtrees in that order (or its only subtree if it is a unary operation).

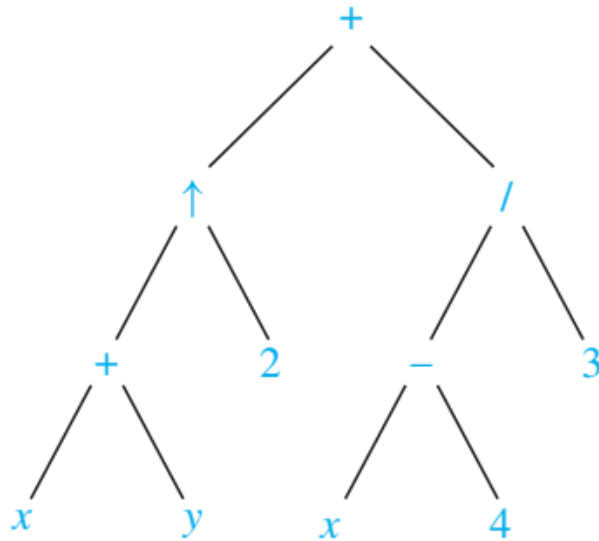
$((x + y) \uparrow 2) + ((x - 4)/3)$



Infix, Prefix, Postfix Notation

⇒ An inorder traversal of a binary tree representing an expression produces the original expression with the elements and operations in the same order as they originally appear, except for unary operation.

But: inorder traversals give ambiguous expressions ⇒ need to include parentheses ⇒ **infix form** (fully parenthesized)

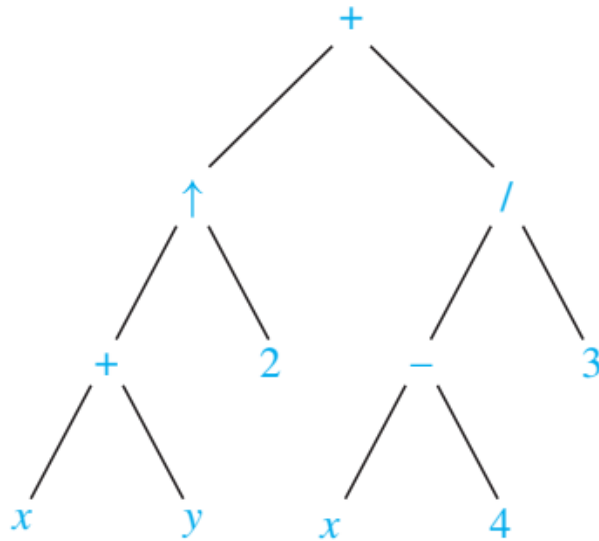


$$((x + y) \uparrow 2) + ((x - 4)/3)$$

Infix, Prefix, Postfix Notation

The **prefix form (Polish notation)** of an expression is obtained by traversing its corresponding rooted tree in preorder.

An expression in prefix form (where each operation has a specified number of operands) is unambiguous.

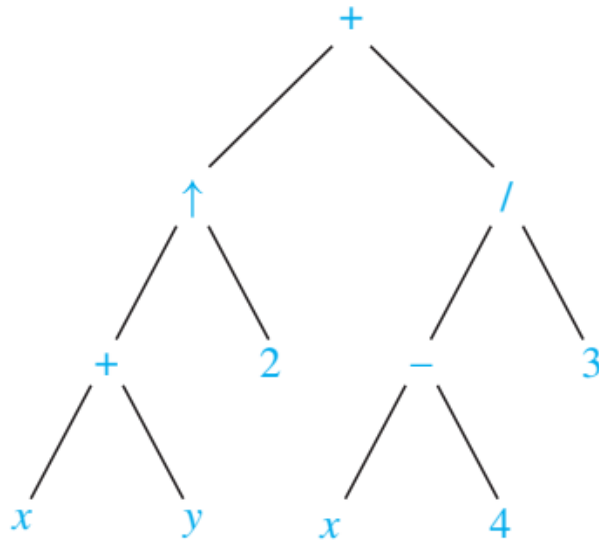


$+ \uparrow + x y 2 / - x 4 3$

- Evaluate an expression in prefix form by working from right to left.
- When we encounter an operator, we perform the corresponding operation with the two operands immediately to the right of this operand.

Infix, Prefix, Postfix Notation

The **postfix form (reverse Polish notation)** of an expression is obtained by traversing its corresponding rooted tree in postorder. An expression in postfix form (where each operation has a specified number of operands) is unambiguous.



$x \ y + 2 \uparrow x \ 4 - 3 / +$

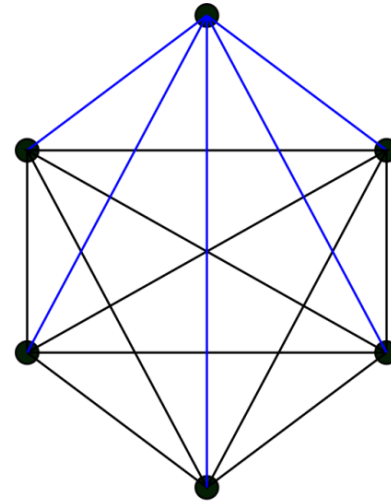
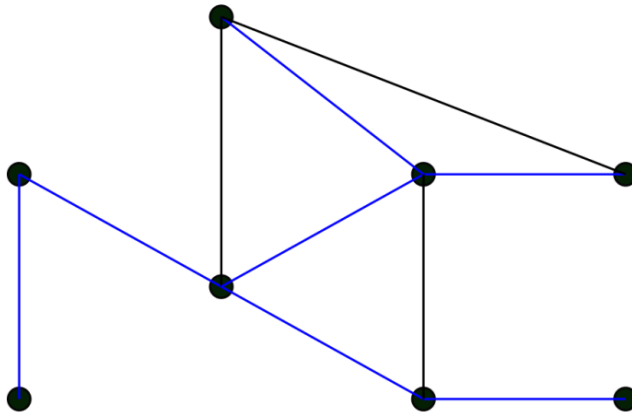
- Work from left to right, carrying out operations whenever an operator follows two operands.
- After an operation is carried out, the result of this operation becomes a new operand.

Spanning Trees

Definition

Let G be a simple graph. A **spanning tree** of G is a subgraph of G that is a tree containing every vertex of G .

Example:



Depth-first Search

Recursive algorithm

DFS(G : connected graph with vertices $v_1, v_2, \dots, v_n$)

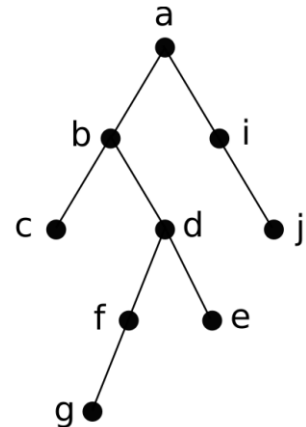
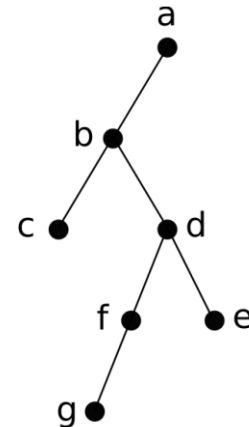
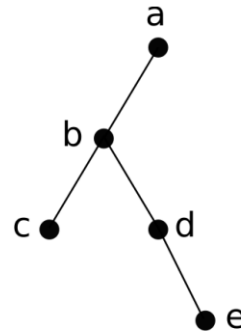
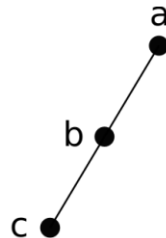
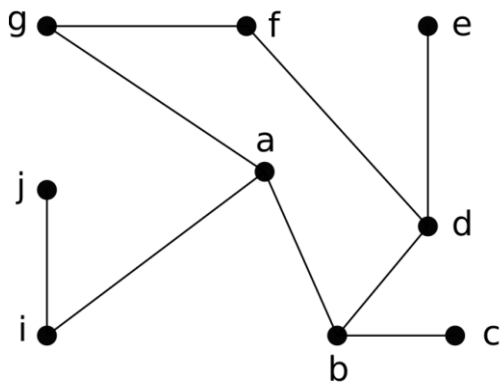
$T :=$ tree consisting only of the vertex v_1

visit(v_1)

visit(v : vertex of G)

for each vertex w adjacent to v and not yet in T
add vertex w and edge (v, w) to T

visit(w)



Breadth-first Search

Algorithm

BFS(G : connected graph with vertices $v_1, v_2, \dots, v_n$)

$T :=$ tree consisting only of vertex v_1

$L :=$ empty list

put v_1 in the list L of unprocessed vertices

while L is not empty

 remove the first vertex v from L

for each neighbour w of v

if w is not in L and not in T **then**

 add w to the end of the list L

 add w and the edge (v, w) to T

